School of Computing and Augmented Intelligence
ASU IRA A. FULTON SCHOOLS OF engineering

CSE 446/598
Software Integration and Engineering

Unit 2
Software Architecture and Integration

Lecture 2-1
Advanced Web App Architecture and Platforms

Dr. Yinong Chen
https://myasucourses.asu.edu/

Unit 2 Outline

Unit 2-1 Advanced Web App Architecture

Unit 2-2 Enterprise Architecture and Business Process

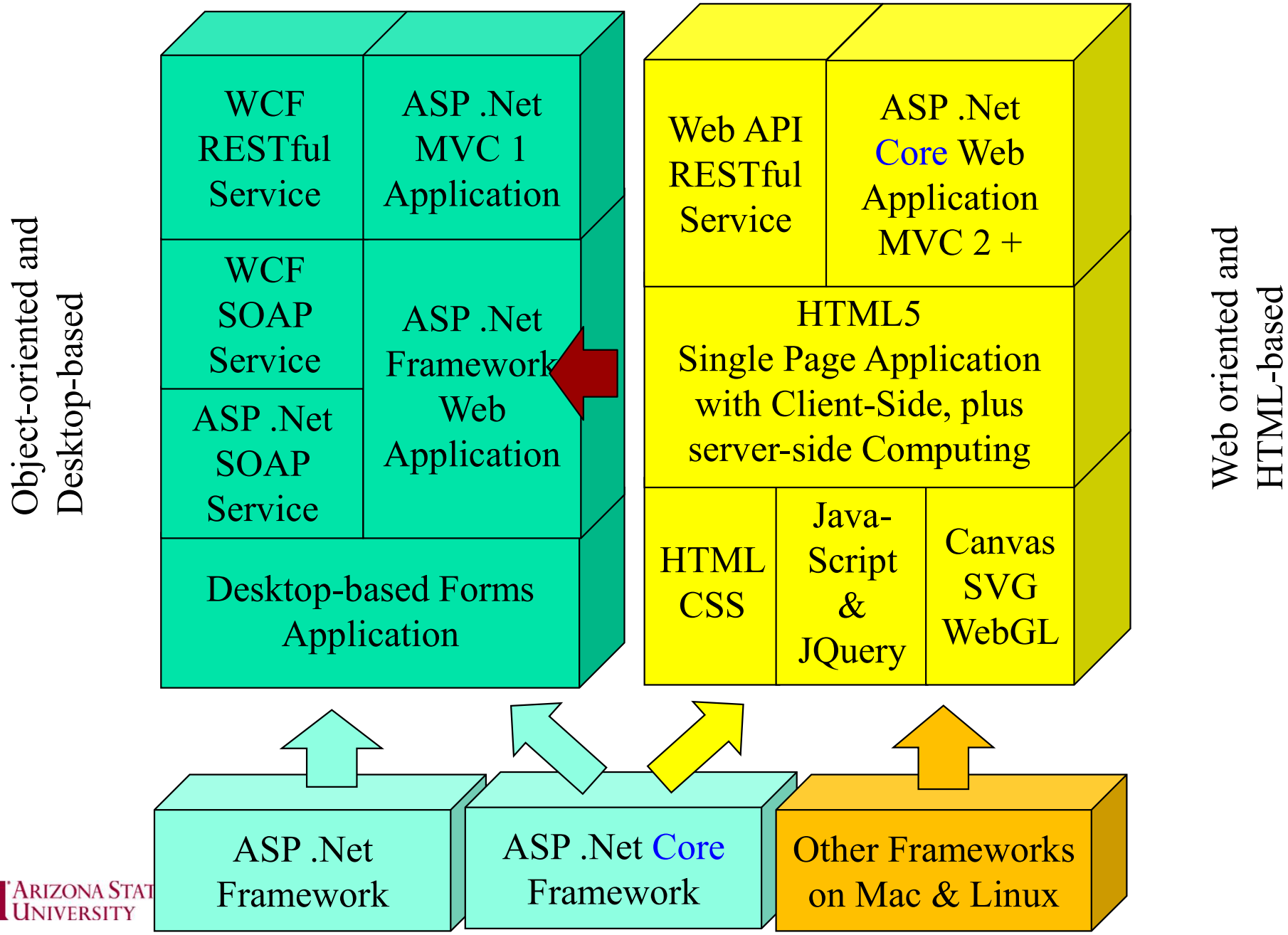
Unit 2-3 Workflow Concepts and Workflow Foundation

Unit 2-4 Software Integration Using Power Platform

Unit 2-1 Lecture Outline

- ASP .Net Web Application Architecture
  - Evolution of Web Technologies from OO to Web
  - ASP .Net (Core) Framework
- HTML5 Application Architecture
  - Platforms
  - Case Study
- MVC Web Application Architecture
  - Platforms
  - Case Study

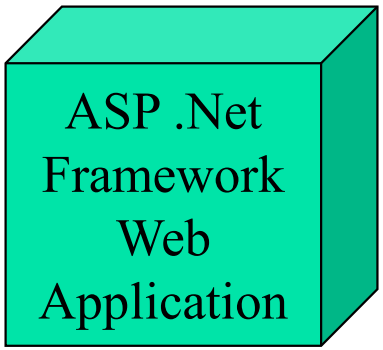
Evolution of Web Technologies: Started from OO

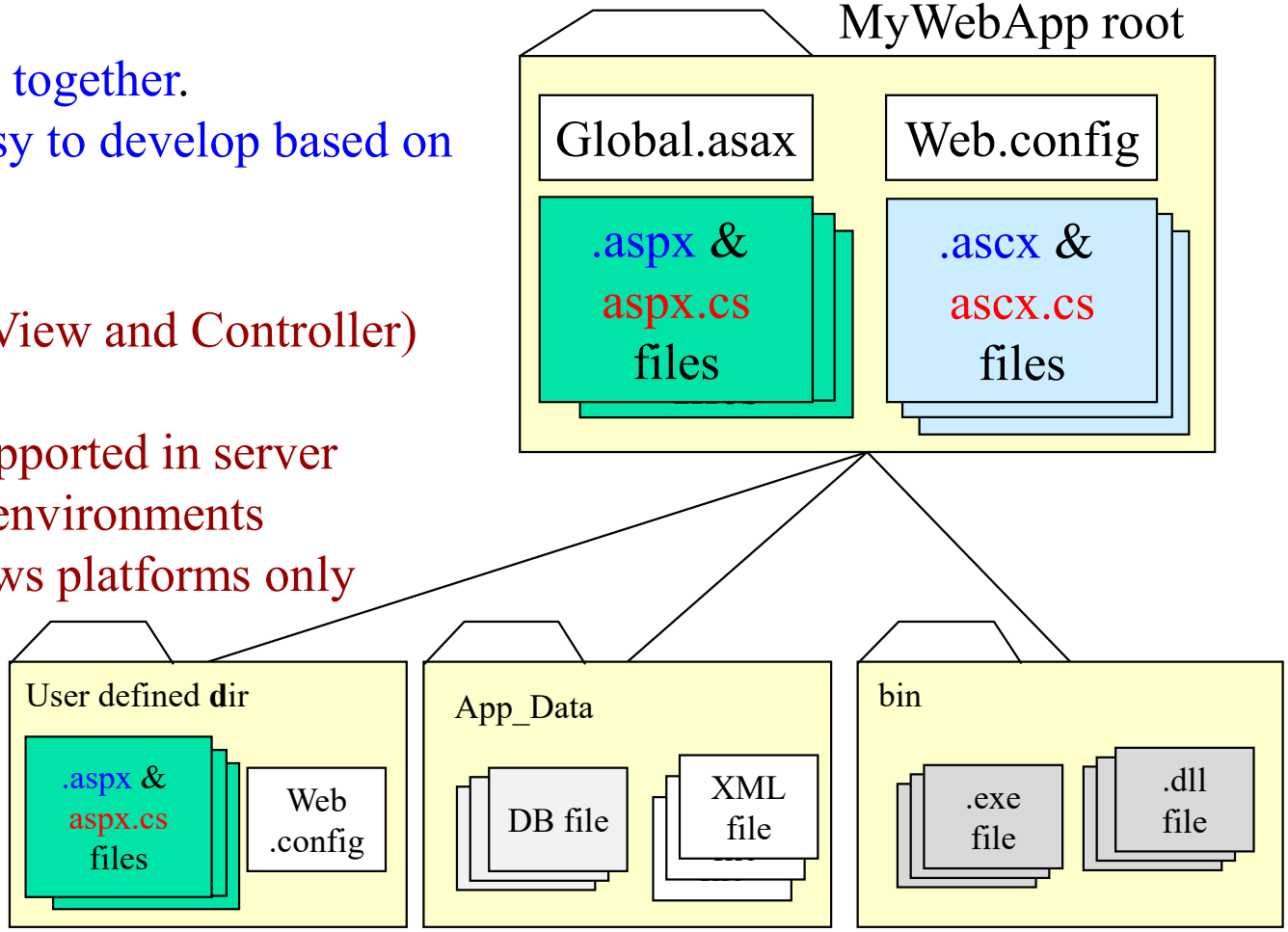


Review: ASP .Net Web Application Architecture

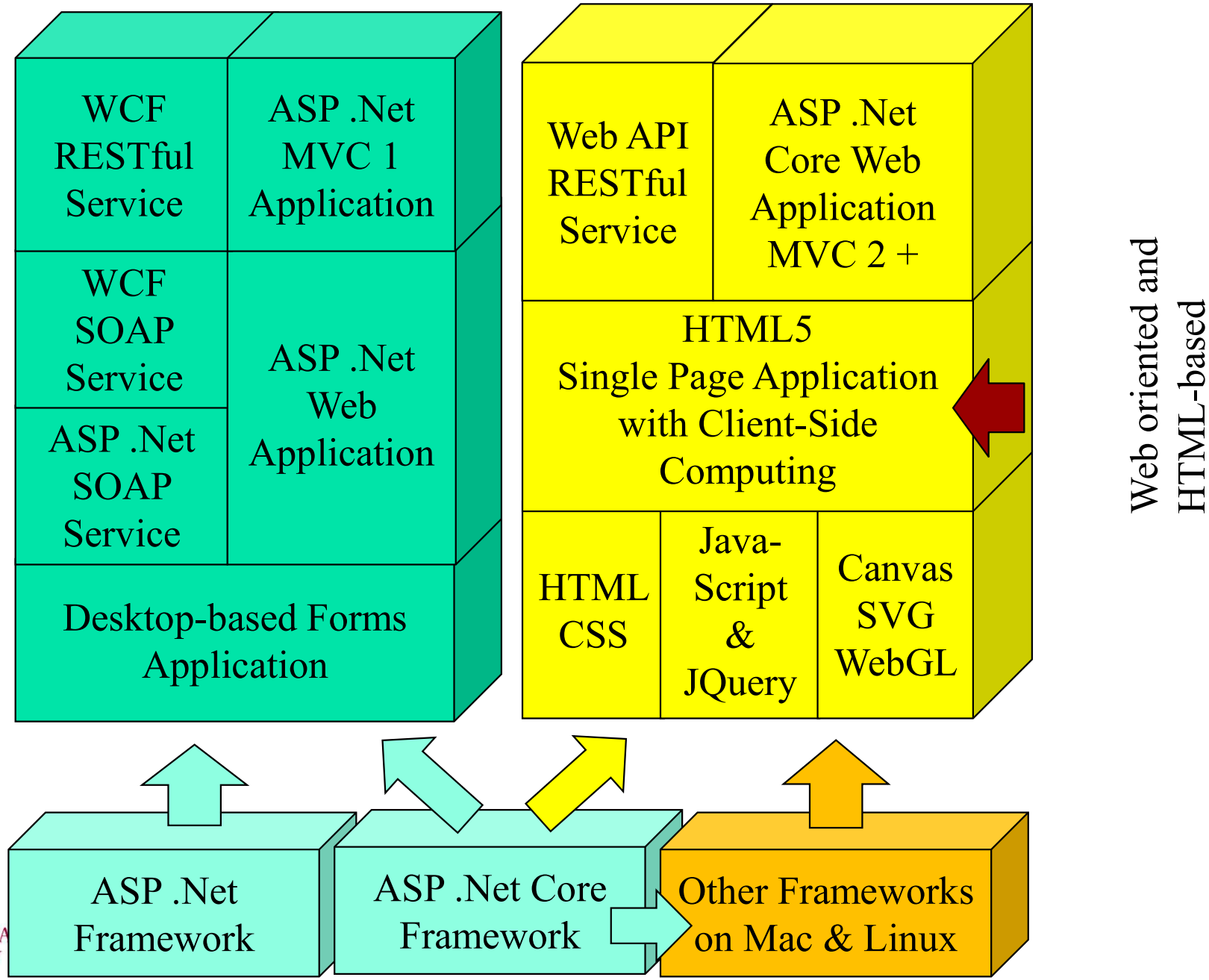
ASP .Net Framework comes from object-oriented programming environment.
It makes it easier for developers with OO development experience.
It places the presentation layer files (.aspx) and the controller file (.aspx.cs) together.
It generates html file based on .aspx file.

- The advantages
  - Related items are placed together.
  - It matches OO and is easy to develop based on OO principles.
- The disadvantages:
  - Mix two tiers together (View and Controller) and add dependency.
  - ViewState is not well supported in server farms/cloud computing environments
  - Development on Windows platforms only





The Evolution Web Technologies: Move to HTML



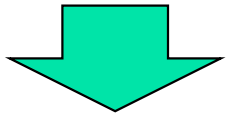
From HTML to HTML5

Reference: http://www.w3schools.com/html/html5_intro.asp

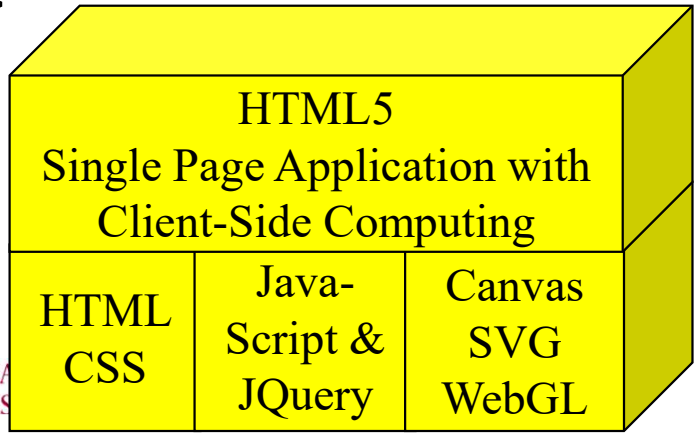
```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title> Doc  Title </title>
</head>

<body>
Content of the document.....
</body>

</html>
```

- New semantic elements like <header>, <footer>, <article>, and <section>.
- New form control attributes like number, date, time, calendar, and range.
- New graphic elements like: <svg> and <canvas>.
- New multimedia elements like: <audio> and <video>.

JavaScript and JQuery API Calls



Become a Web Application Development Language

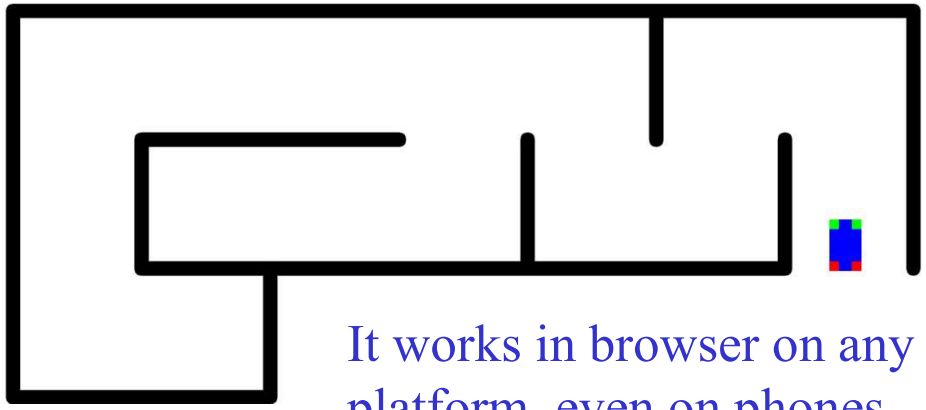


HTML5 JavaScript API Library

Reference: http://html5index.org/

- Web communication protocols
  - Web Sockets (Connection-oriented)
  - Messaging, and
  - WebRTC
    (Web Real-Time Communication)
- Drag and Drop, Fullscreen
- Canvas, SVG, WebGL
- Animation Timing, Media, Pointer Lock, Web Audio
- File API, File System API, Indexed DB, Offline, Web Storage
- Browser, Shadow DOM, Typed Arrays, Web Workers
- DOM and JQuery
- CSS Object Model, Selectors

Example: Robotics Simulator on Web
http://venus.sod.asu.edu/VIPLE/Websimulator/



It works in browser on any platform, even on phones.

Canvas, SVG, or WebGL? Choose the right API

https://msdn.microsoft.com/en-us/library/dn265058%28v=vs.85%29.aspx?f=255&MSPPError=-2147217396

- Canvas graphic library is the typical choice for most HTML5 applications. It's simple and speedy, particularly for games with many objects.
- SVG: While Canvas provides simplicity and speed, SVG provides flexibility. For example: Each graphic object is part of the DOM tree for flexible access, and each graphic object in the application can have one or more associated event handlers for processing.
- WebGL: If you are already familiar with OpenGL, you *can* use WebGL for a simple 2D application. If you are not, this API is more complex than you need for a simple graphic application.

Creating HTML5 Applications

- **Using Visual Studio**
  - For HTML5 application development
- Using NetBeans
  - For HTML5 application development

HTML5 Applications in Visual Studio 2019/2022

Create a new project

web

Recent project templates

C# Windows Web

- ASP.NET Core Web Application C#
- Empty Project C++
- Blank App (Universal Windows - C++/CX) C++
- Console App (.NET Core) C#
- WCF Service Application C#

ASP.NET Core Web Application

Project templates for creating ASP.NET Core web apps and web APIs for Windows, Linux and macOS using .NET Core or .NET Framework. Create web apps with Razor Pages, MVC, or Single Page Apps (SPA) using Angular, React, or React + Redux.

C# Linux macOS Windows Cloud Service Web

ASP.NET Web Application (.NET Framework)

Project templates for creating ASP.NET applications. You can create ASP.NET Web Forms, MVC, or Web API applications and add many other features in ASP.NET.

C# Windows Cloud Web

Configure your new project

ASP.NET Web Application (.NET Framework) C# Windows Cloud Web

Project name

Html5App

Location

C:\Users\ychen10\Dropbox (ASU)\Current in Dropbox\Teach All\CSE445-446 Programs\2020\ ...

Choose Single Page Application (HTML5)

Create a new ASP.NET Web Application

Empty

An empty project template for creating ASP.NET applications. This template does not have any content in it.

Web Forms

A project template for creating ASP.NET Web Forms applications. ASP.NET Web Forms lets you build dynamic websites using a familiar drag-and-drop, event-driven model. A design surface and hundreds of controls and components let you rapidly build sophisticated, powerful UI-driven sites with data access.

MVC

A project template for creating ASP.NET MVC applications. ASP.NET MVC allows you to build applications using the Model-View-Controller architecture. ASP.NET MVC includes many features that enable fast, test-driven development fo creating applications that use the latest standards.

Web API

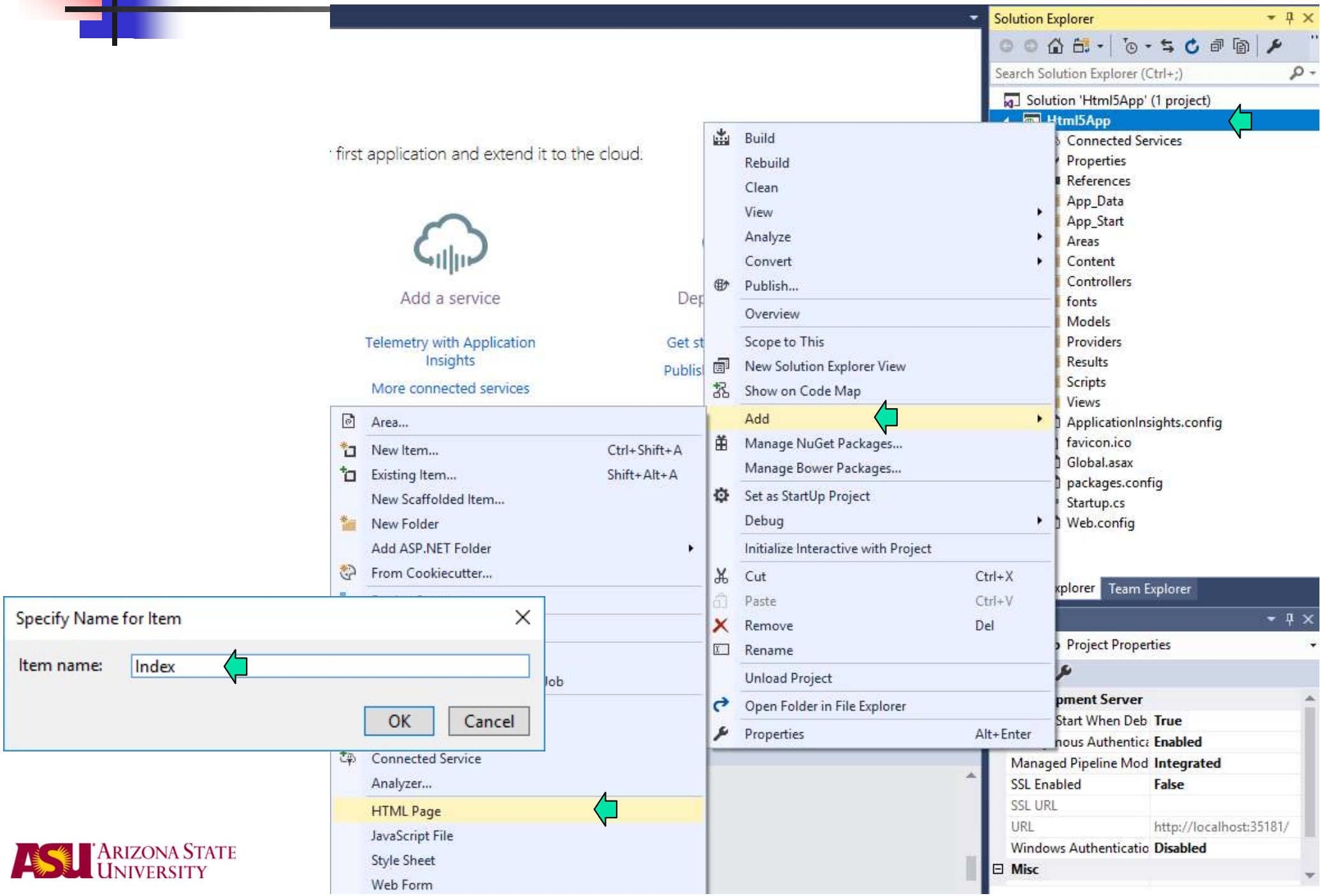
A project template for creating RESTful HTTP services that can reach a broad range of clients including br mobile devices.

Single Page Application

A project template for creating rich client side JavaScript driven HTML5 applications using ASP.NET Web API. Single Page Applications provide a rich user experience which includes client-side interactions using HTML5, CSS3, and JavaScript.

HTML5 app is considered a special case of MVC, with a single page only!

Adding an HTML Page as the Index Page



Index.html Page Created

Index.html | Html5App

```
1   <!DOCTYPE html>
2   <html>
3   <head>
4       <meta charset="utf-8" />
5       <title></title>
6   </head>
7   <body>
8
9   </body>
10  </html>
```

146 %

Solution Explorer

Search Solution Explorer (Ctrl+;)

- Solution 'Html5App' (1 project)
  - Html5App
    - Connected Services
    - Properties
    - References
    - App_Data
    - App_Start
    - Areas
    - Content
    - Controllers
    - fonts
    - Models
    - Providers
    - Results
    - Scripts
    - Views
    - ApplicationInsights.config
    - favicon.ico
    - Global.asax
    - Index.html
    - packages.config
    - Startup.cs
    - Web.config

Hello HTML5

- Right Click and choose View in Browser

Index.html | Html5App

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title>Hello</title>
</head>
<body>
    Hello HTML5
</body>
</html>
```

146 %

Solution Explorer

Search Solution Explorer (Ctrl+;)

- App_Start
- Areas
- Content
- Controllers
- fonts
- Models
- Providers
- Results
- Scripts
- Views
- ApplicationInsights.config
- favicon.ico
- Global.asax
- Index.html
- packages.config
- Startup.cs
- Web.config

Solution Explorer | Team Explorer

Right Click, choose View in Browser

localhost:35181/Index.html

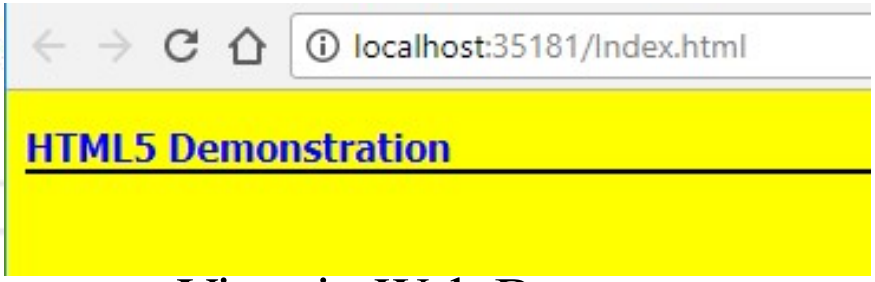
Hello HTML5

Adding CSS Stylesheet to HTML5 Page

- Right Click Project → Add → Style Sheet
- Name the page StyleSheet1, and link it into Index.html page

StyleSheet1.css | Index.html

```
1  body {
2      font-family: Tahoma, Arial, sans-serif;
3      font-size: 14px;
4      color: blue;
5      background: yellow;
6      margin: 8px;
7  }
8
9  h1 {
10     font-
11     margi
12     margi
13     borde
14 }
```



View in Web Browser

StyleSheet1.css | Index.html

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <meta charset="utf-8" />
5      <link rel="stylesheet" href="StyleSheet1.css">
6      <title>Hello</title>
7  </head>
8  <body>
9      <h1>HTML5 Demonstration</h1>
10 </body>
11 </html>
```

GUI Design

Toolbox

Search Toolbox

HTML

- Pointer
- Input (Button)
- Input (Reset)
- Input (Submit)
- Input (Text)
- Input (File)
- Input (Password)
- Input (Checkbox)
- Input (Radio)
- Input (Hidden)
- Textarea
- Table
- Image

StyleSheet1.css | Index.html

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <meta charset="utf-8" />
5      <link rel="stylesheet" href="StyleSheet1.css">
6      <title>Hello</title>
7  </head>
8  <body>
9      <h1>HTML5 Demonstration</h1>
10     <textarea id="TextArea1" rows="2" cols="40">
11 This demo shows the GUI Design and Code behind the GUI
12     </textarea><br />
13     <input id="Text1" type="text" />
14     <input id="Button1" type="button" value="button" /><br /
15     <input id="Submit1" type="submit" value="submit" />
16 </body>
17 </html>
```

Solution Explorer

Search Solution Explorer (Ctrl+;)

- Areas
- Content
- Controllers
- fonts
- Models
- Providers
- Results
- Scripts
- Views
- ApplicationInsights.confi
- favicon.ico
- Global.asax
- Index.html
- packages.config
- Startup.cs

localhost:35181/Index.html

HTML5 Demonstration

This demo shows the GUI Design and Code behind the GUI

button

submit

View in Web Browser

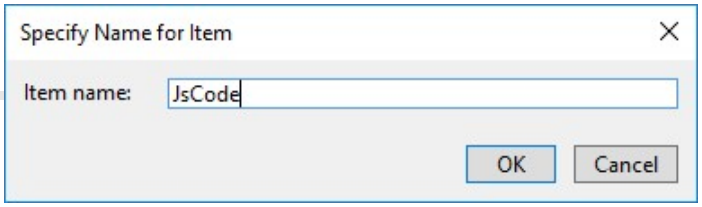
There is no "Design" view in HTML GUI design

Adding JavaScript Code in HTML

Index.html

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <link rel="stylesheet" href="StyleSheet1.css">
    <title>Hello</title>
</head>
<body>
    <script type="text/JavaScript">
    function add2Nos() {
        document.sum.z.value =
            parseInt(document.sum.x.value) + parseInt(document.sum.y.value)
    }
    </script>

    <h1>HTML5 Demonstration</h1>
    <textarea id="TextArea1" rows="2" cols="40">
This demo shows addition operation in JS
    </textarea><br />
    <form name="sum">
        <input type="text" name="x" size=5 maxlength="5">
        +
        <input type="text" name="y" size="5" maxlength="5">
        <input type="button" value=" = " name="Submit" onClick="add2Nos()">
        <input type="text" name="z" size="5" maxlength="5">
        <span id="result"></span>
    </form>
</body>
</html>
```

Adding a JavaScript File

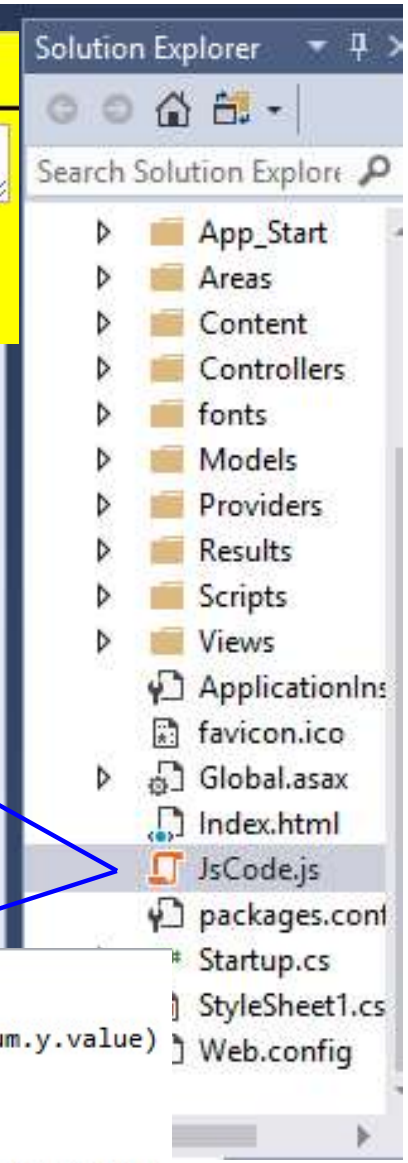


- Right Click Project → Add → JavaScript File
- Name the file JsCode, and link it into Index.html page

Index.html*

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <link rel="stylesheet" href="StyleSheet1.css">
    <script src="JsCode.js"></script>
    <title>Hello</title>
</head>
<body>
    <h1>HTML5 Demonstration</h1>
    <textarea id="TextArea1" rows="2" cols="40">
This demo shows JavaScript in a external file
    </textarea><br />
    <form name="sum">
        <input type="text" name="x" size=5 maxlength="5">
        and
        <input type="text" name="y" size="5" maxlength="5">
        <input type="button" value=" add " name="Submit" onClick="add2Nos()">
        <input type="button" value=" multiply " name="Submit" onClick="multiply2Nos()">
        <br />
        =
        <br />
        <input type="text" name="z"
        <span id="result"></span>
    </form>
</body>
</html>
```

View in Web Browser

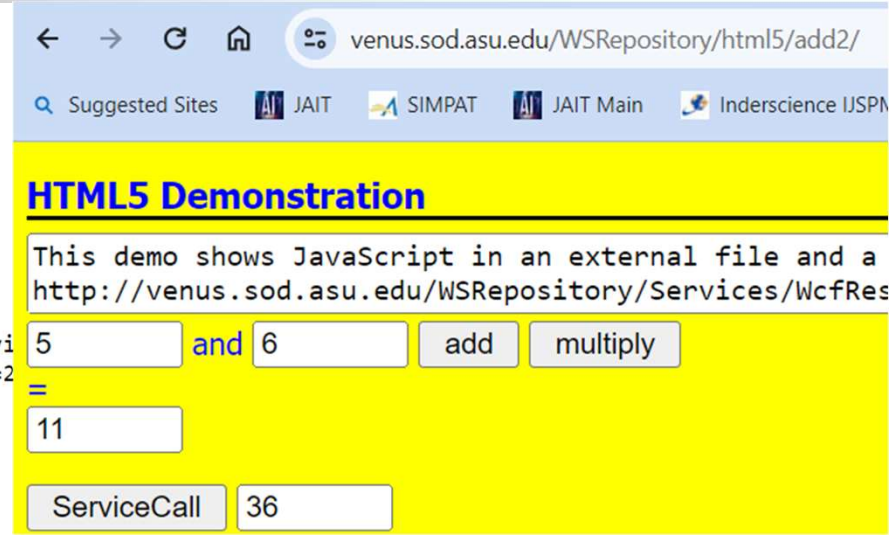



```
function add2Nos() {
    document.sum.z.value =
        parseInt(document.sum.x.value) + parseInt(document.sum.y.value)
}
function multiply2Nos() {
    document.sum.z.value =
        parseInt(document.sum.x.value) * parseInt(document.sum.y.value)
}
```

Calling a RESTful Service in JS

https://venus.sod.asu.edu/WSRepository/html5/add2/

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <link rel="stylesheet" href="StyleSheet1.css">
    <script src="JsCode.js"></script>
    <title>HTML5 Demo</title>
</head>
<body>
    <h1>HTML5 Demonstration</h1>
    <textarea id="TextArea1" rows="2" cols="100">
This demo shows JavaScript in an external file and a RESTful service call to the servi
https://venus.sod.asu.edu/WSRepository/Services/WcfRestService4/Service1/add2?x=11&y=2
    </textarea><br />
    <form name="sum">
        <input id="Text1" type="text" name="x" size=5 maxlength="5" />
        and
        <input id="Text2" type="text" name="y" size="5" maxlength="5" />
        <input type="button" value=" add " name="Submit" onClick="add2Nos()" />
        <input type="button" value=" multiply " name="Submit" onClick="multiply2Nos()" />

        <br />
        =
        <br />
        <input type="text" name="z" size="5" maxlength="5">
        <span id="result"></span><p />
        <input type="button" value=" ServiceCall " name="Submit" onClick="ServiceCall()" />
        <input type="text" name="s" size="5" maxlength="5">
        <span id="result"></span>

    </form>
<script defer src="https://static.cloudflareinsights.com/beacon.min.js/v84a3a4012de94ce1a686
</body>
</html>
```




```
function multiply2Nos() {
    document.sum.z.value =
        parseInt(document.sum.x.value) * parseInt(document.sum.y.value)
}
```

```
function ServiceCall() {
    var xhttp = new XMLHttpRequest();
    xhttp.onreadystatechange = function () {
        if (xhttp.readyState == 4 && xhttp.status == 200)
            document.sum.s.value = xhttp.responseText;
    }
    xhttp.open("GET", "https://venus.sod.asu.edu/WSRepository/Services/WcfRestService4/Service1/add2?x=11&y=25", true);
    xhttp.setRequestHeader("Content-type", "application/json");
    xhttp.send();
}
```

The ServiceCall is asynchronous:

(1) Start request asynchronously, not block waiting here; Waiting for WS result;

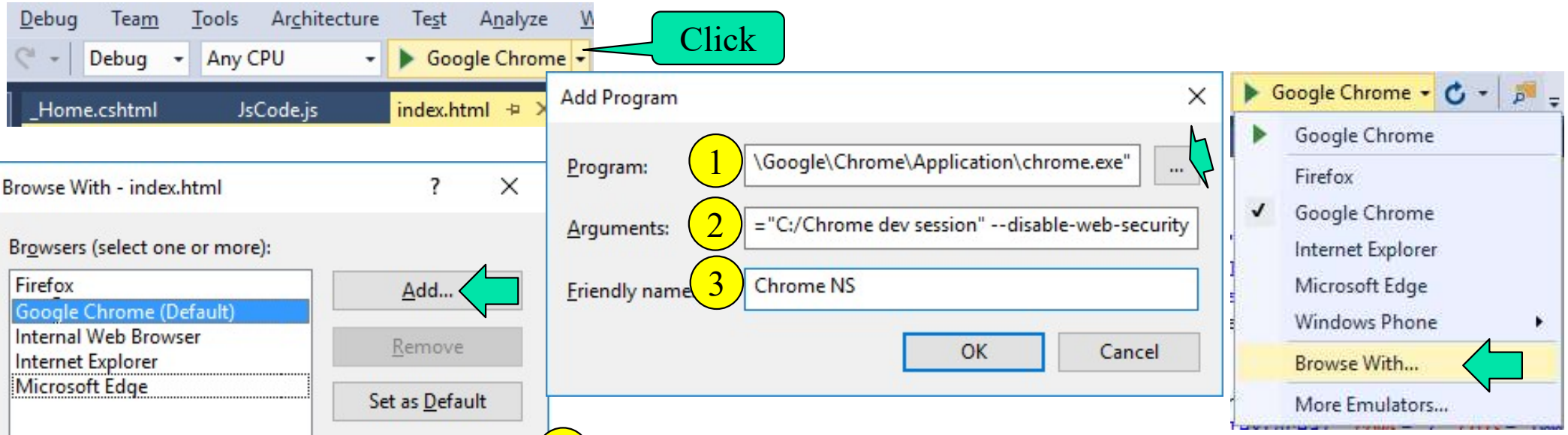
(2) Call the service and obtain the result;

(3) Send the result as return value

You could do other work here

W3C: Cross-Origin Resource Sharing (CORS)

- **Testing**: Disable Web browser security for testing XMLHttpRequest in Visual Studio



2 --user-data-dir="C:/Chrome dev session" --disable-web-security

https://chromewebstore.google.com/detail/allow-cors-access-control/lhobafahddgcelffkeicbaginigeejlf

- Browser security policy prevents a web page from making AJAX requests to another domain.

RESTful Service / Web API ← CORS request / XMLHttpRequest — Web Client HTML

Balance of capacity & security

- CORS is a W3C standard that allows a server to relax the same-origin policy (https://www.w3.org/TR/cors/).
- **Deployment**, follow: https://docs.microsoft.com/en-us/aspnet/web-api/overview/security/enabling-cross-origin-requests-in-web-api

CORS on IIS

https://enable-cors.org/server_iis7.html

- If you are receiving a service or AJAX call error, try to add the following element in your Web.config file:

```
<?xml version="1.0" encoding="utf-8"?>
<configuration>
    <system.webServer>
        <httpProtocol>
            <customHeaders>
                <add name="Access-Control-Allow-Origin" value="*" />
            </customHeaders>
        </httpProtocol>
    </system.webServer>
</configuration>
```

For more detail, read:
https://docs.microsoft.com/en-us/iis/extensions/cors-module/cors-module-configuration-reference#cors-configuration

Calling A RESTful Service in jQuery

Source: https://spring.io/guides/gs/consuming-rest-jquery/

```html
<!DOCTYPE html>
<html>
<head>
    <title>Hello jQuery</title>
    <script src="https://ajax.googleapis.com/ajax/libs/jquery/1.10.2/jquery.min.js"></script>
    <script src="hello.js"></script>
</head>

<body>
    <div>
        <p class="greeting-id">The ID is </p>
        <p class="greeting-content">The content is </p>
    </div>
</body>
</html>
```

Warning: Making service calls in html5 is possible but is restricted for security reason. It is better to make service calls on server-side using MVC architecture.

```javascript
$(document).ready(function () {
    $.ajax({
        url: "http://rest-service.guides.spring.io/greeting"
    }).then(function (data) {
        $('.greeting-id').append(data.id);
        $('.greeting-content').append(data.content);
    });
});
```

// hello.js

Making asynchronous call in jQuery

http://venus.sod.asu.edu/WSRepository/html5/hello/

The ID is 37

The content is Hello, World!

HTML5 Example with Animation Definition

```html
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title>first website</title>
    <link href="main.css" rel="stylesheet" />
    <script src="main.js"></script>
</head>
<body>
    <div id="domino">
        <h1>enter an animal</h1>
        <input type ="text" id="animal">
        <button onclick="createanimal()">create animal</button>
        <button onclick="displayanimal()">display animal</button>
        <p id="show"></p>

    </div>
</body>
</html>
```

```javascript
var animallist = [];
var item;
currentlist = [];
function createanimal()
{

    item = document.getElementById("animal").value;
    animallist.push(item);
    console.info("animal " + animallist);
}

function displayanimal()
{
    for (var i = 0; i < animallist.length; i++)
    {

        var currentitem = animallist[i];
        currentlist[i] = currentitem;
        document.getElementById("show").innerHTML = currentlist;


    }
}
```

Where will the JS code be executed?

(A) They will be executed in the browser.

(B) They will be executed in the server.

(C) They will be executed in the browser if they are embedded in the HTML file.

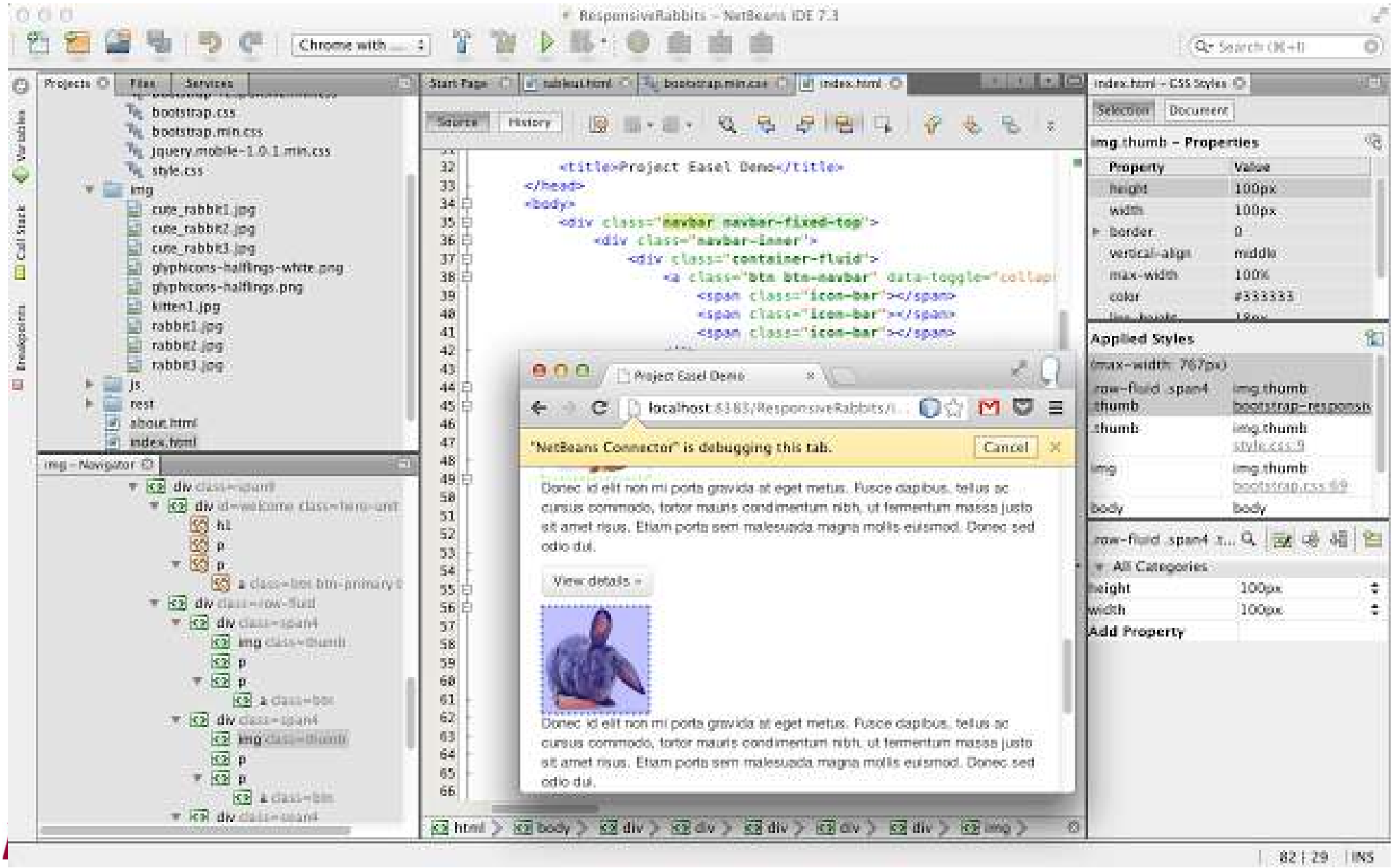
(D) They will be executed in the server if they are in a separate .js file linked to the HTML file.

Preparation for NetBeans

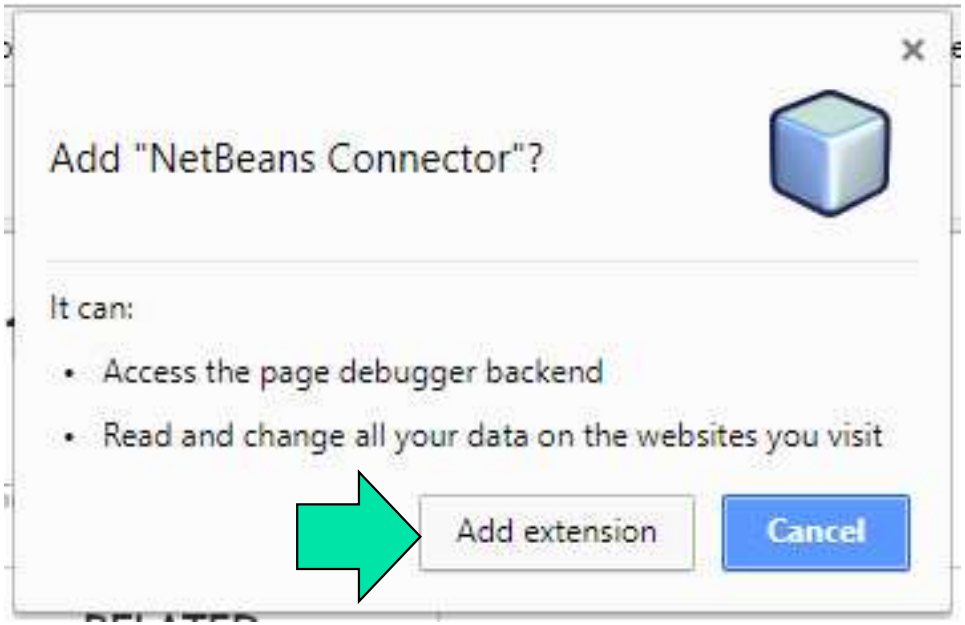
- Please check if the following recourses have been installed/included:
  - NetBeans IDE (pick either HTML5/JavaScriptpackage or all package included) https://netbeans.org/downloads/index.html
  - Java Development Kit (JDK) version 7 or 8 (NetBeans 8.2 does not run JDK9!) or later version.
  - Chrome Browser
- Add NetBeans Connector Extensions into Chrome Browser:
  - Go to the Chrome browser (https://chrome.google.com/webstore/detail/netbeansconnector/hafdlehgocfcodbgjnpecfajgkeejnaa?utm_source=chrome-ntp-icon) and click Add the NetBeans Connector page. Click Add when you are prompted to confirm that you want to add the extension

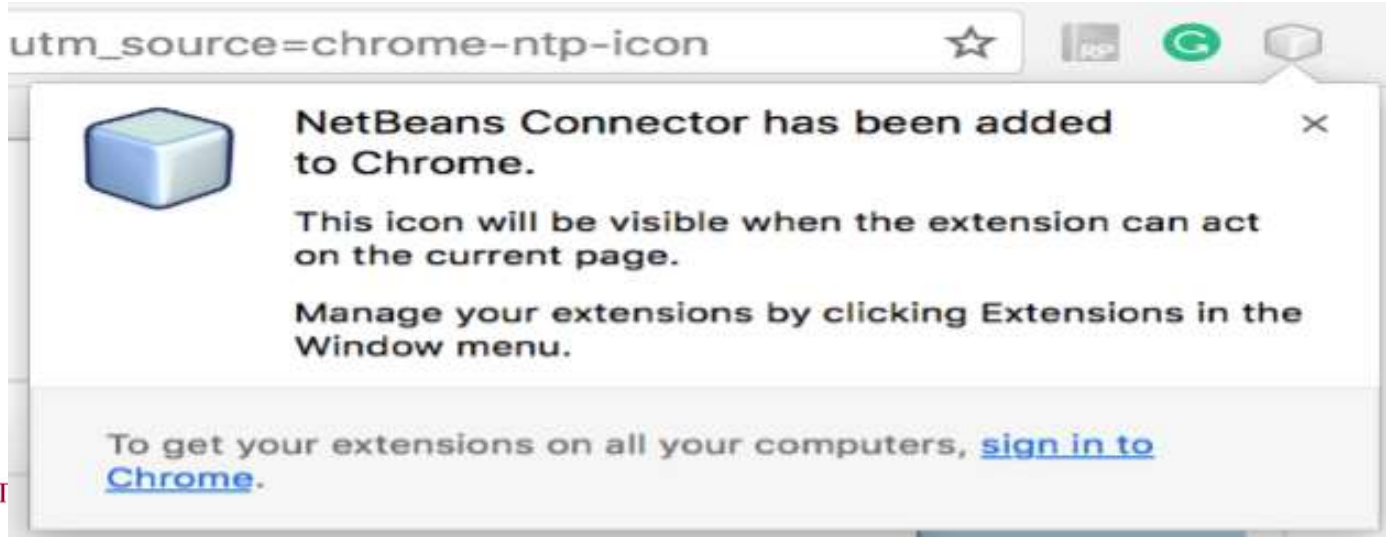
NetBeans Connector

https://chrome.google.com/webstore/detail/netbeansconnector/hafdlehgocfcodbgjnpecfajgkeejnaa?utm_source=chrome-ntp-icon

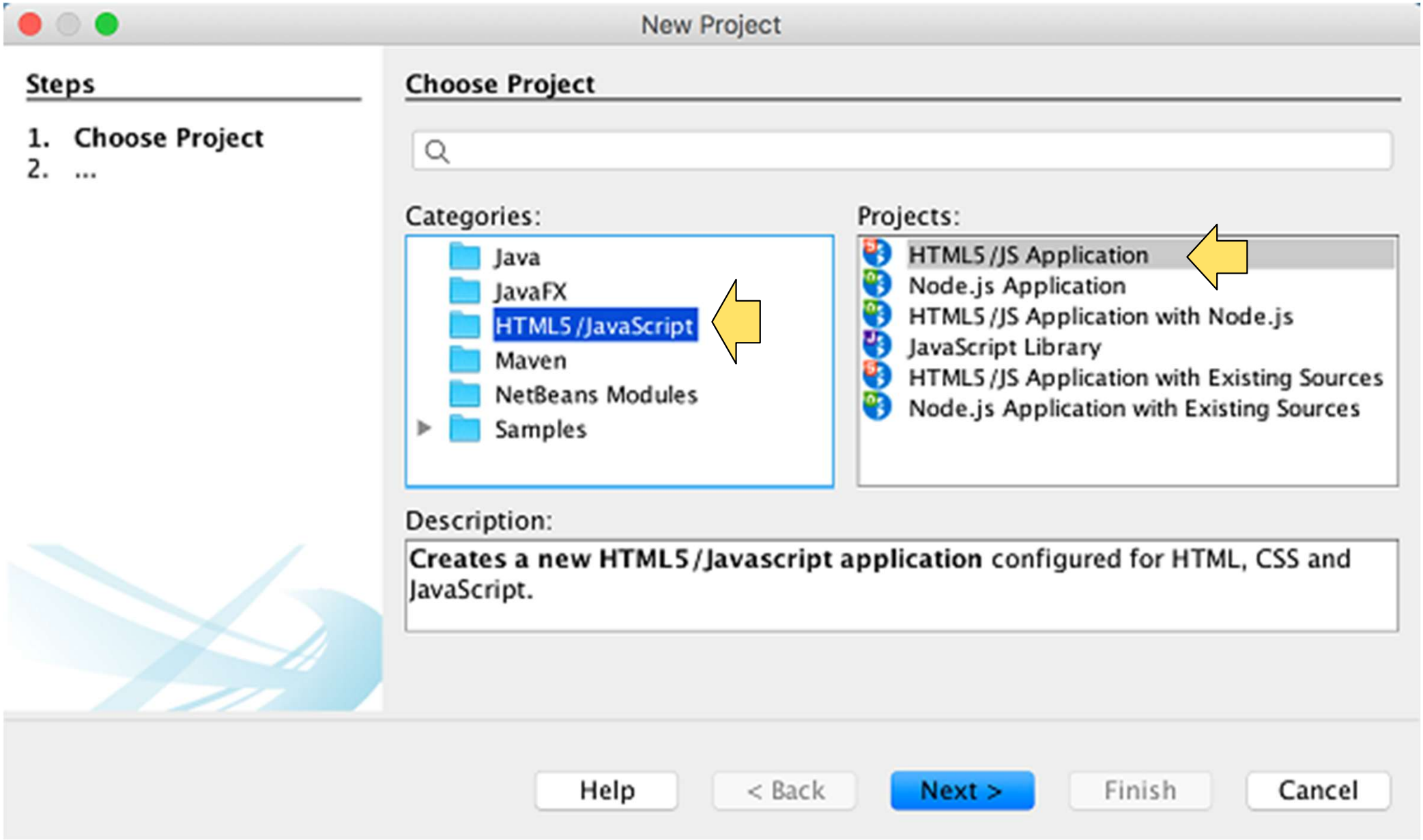


Add to Chrome





Now, Start NetBeans IDE

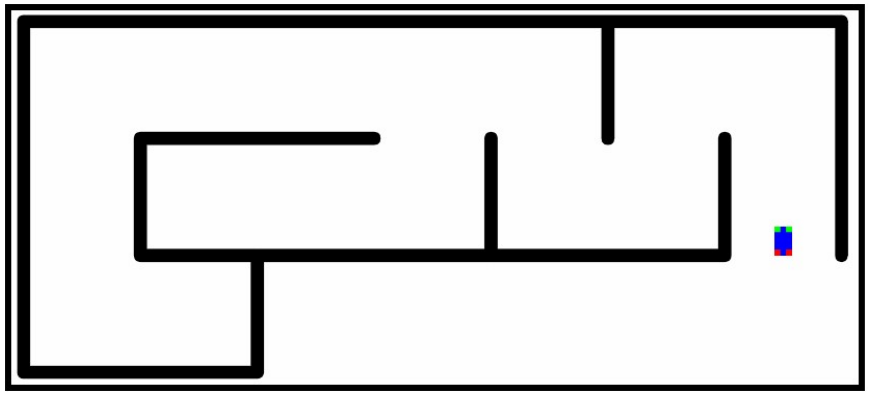


Creating HTML5 Applications

- Using Visual Studio
- **NetBeans**
  - was originally used for developing object-oriented software
  - was extended for developing web services
  - can also be used for developing HTML5 web applications

HTML5 Case Study: ASU IoT Maze Simulator

http://venus.sod.asu.edu/VIPLE/Web2DSimulator/



Add a New Line | Remove a Line

Default: Forward

1. If Right sensor > 100 pixels

Then: Delayed Turn Right by 90 pixels

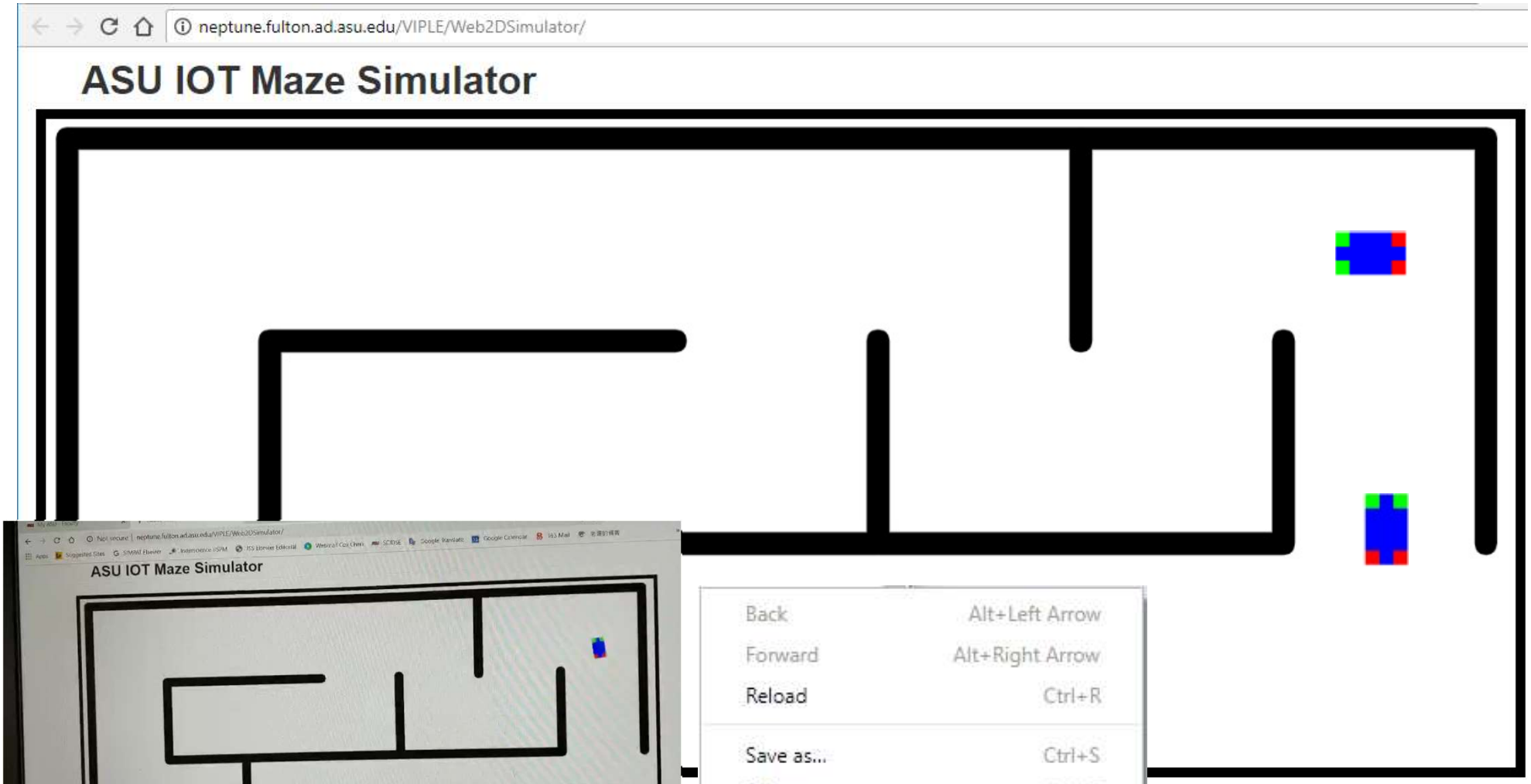
2. Else if Forward sensor <= 50 pixels

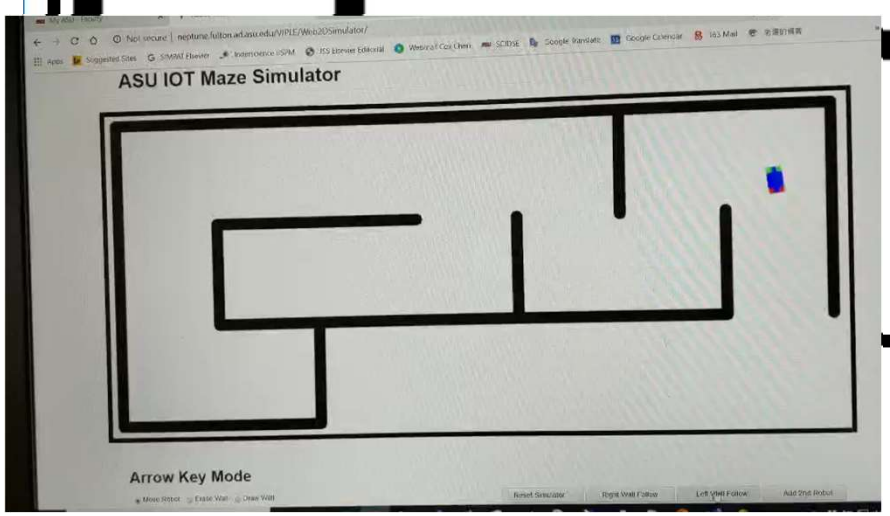
Then: Turn Left

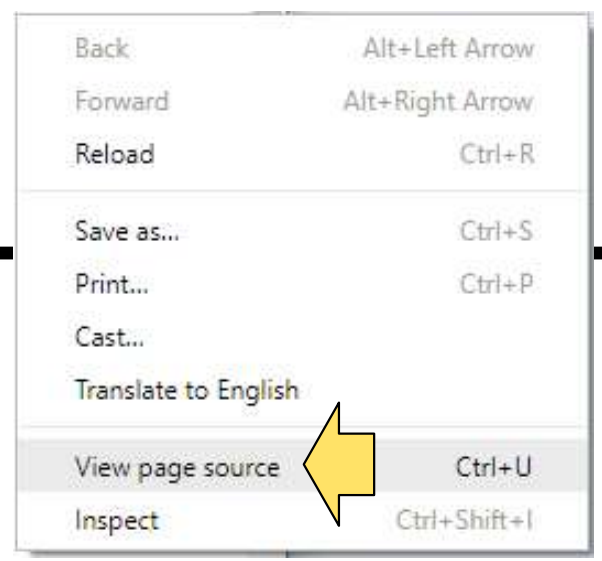
Run

HTML5 Case Study: ASU IOT Maze Simulator

http://venus.sod.asu.edu/VIPLE/Web2DSimulator/







Right-click the page

Study the source code to learn Canvas-based animation

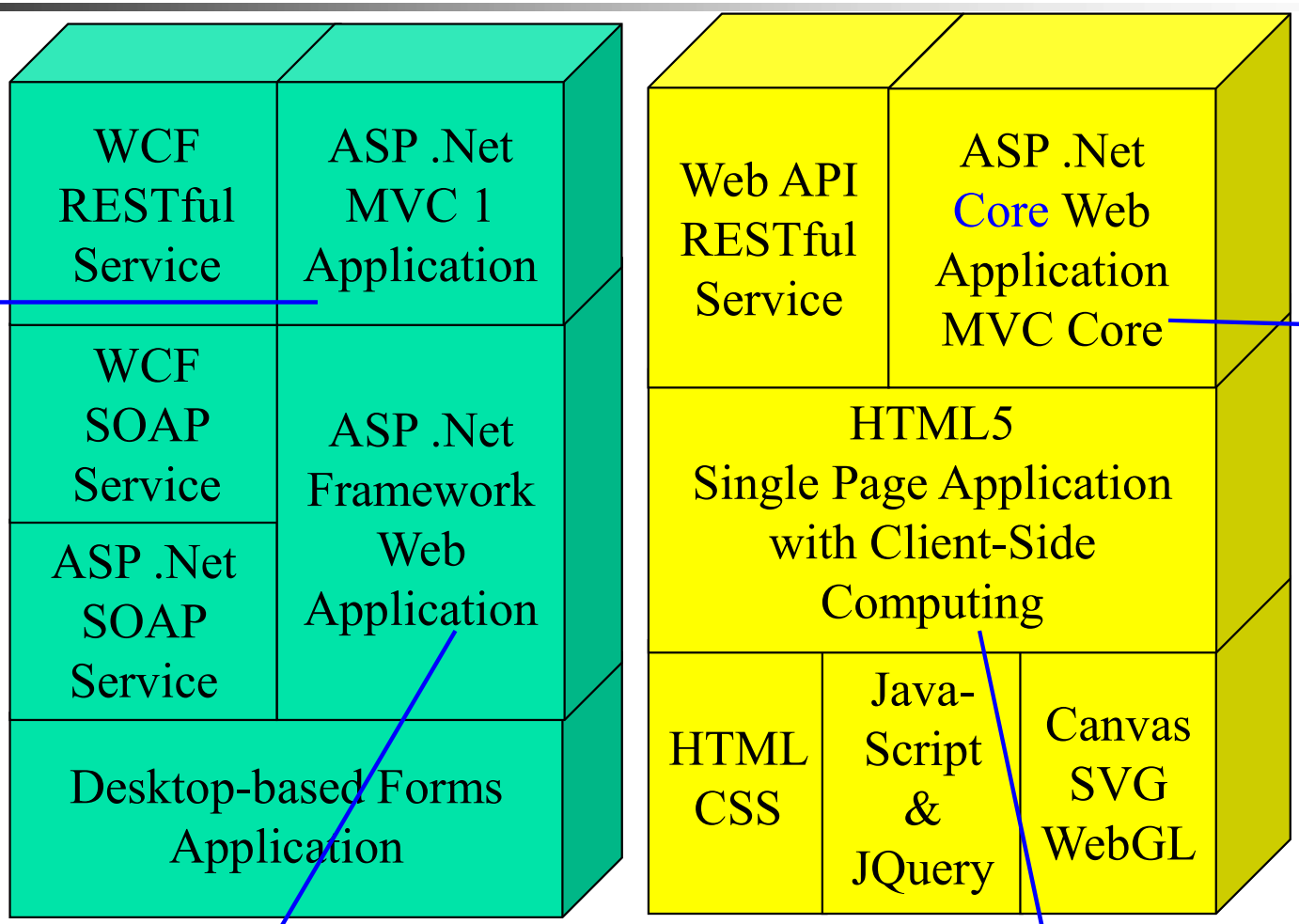
Y. Chen

HTML5 References from MSDN

- Creating User Interface Elements with Canvas or SVG
  https://msdn.microsoft.com/en-us/library/gg589494(v=vs.85).aspx#creating_UI_elements_drawingUI
- Introducing the HTML5 Canvas
  https://msdn.microsoft.com/en-us/magazine/hh708752.aspx

Why MVC Architecture

ASP .Net MVC 1 is a simple re-organization of ASP .Net Web Application.



MVC Core addresses all these issues

- ASP .Net Web Application mixes two tiers together and add dependency.
- ViewState is not well supported in server farms/cloud computing

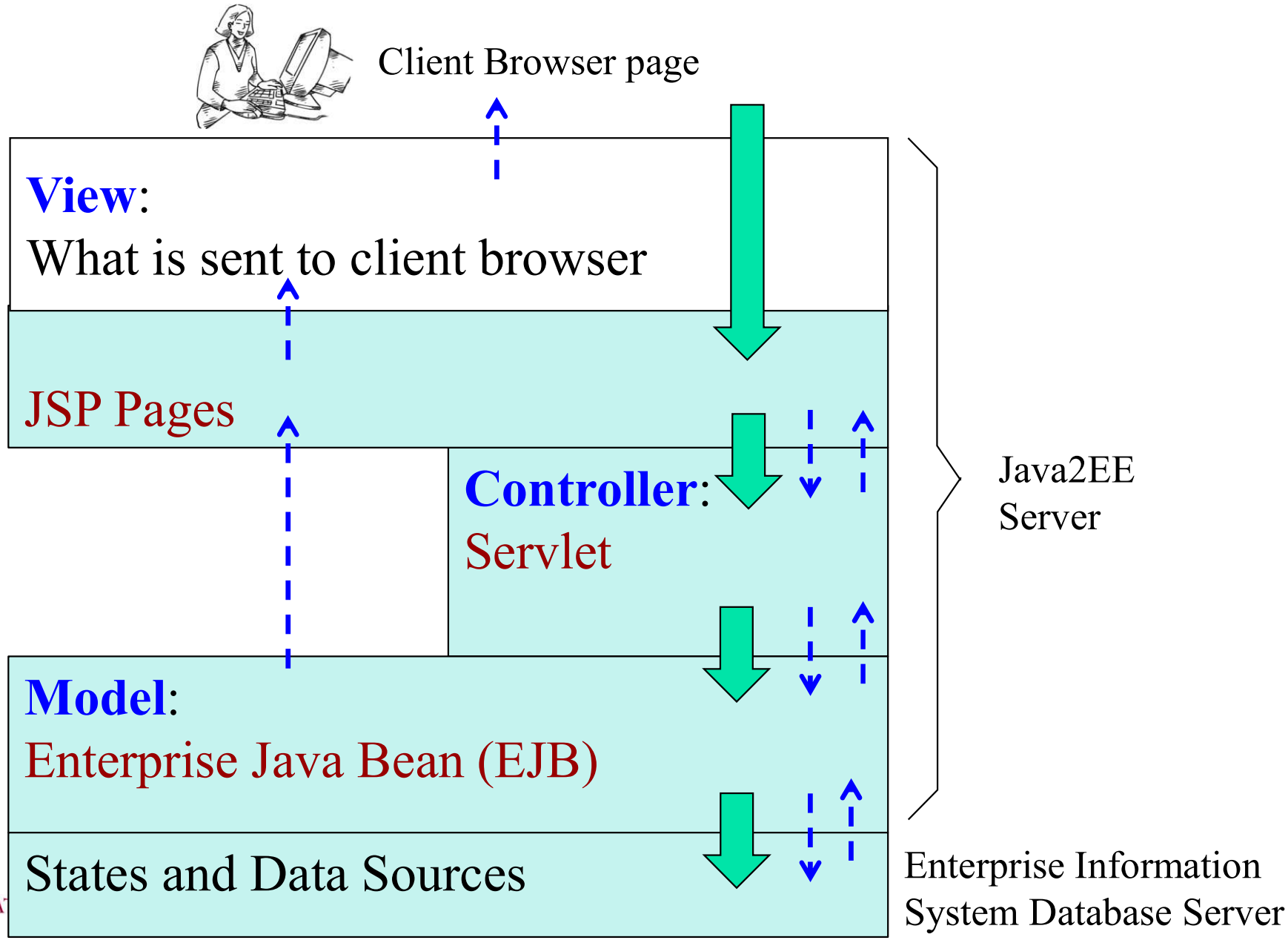
- HTML5 is mainly for one-page application. Service calls are restricted.
- Data are hard to transfer between pages.
- Code viewable: Proprietary and security issues

MVC as an Architecture Pattern

Patterns
- Hardware Patterns
- Software Patters
  - Architecture Patterns (Client-server, tiered, MVC)
  - Computing Patterns (Computing paradigms)
  - Design Patterns (Creational, structural, behavioral, and currency patterns)

- The MVC pattern was originally formulated in the late 1970s by Trygve Reenskaug at Xerox PARC, as part of the Smalltalk. [http://en.wikipedia.org/wiki/Model%E2%80%93view%E2%80%93controller].
- In MVC, the system is divided into three groups of components:
  - **M**odel: contains the components representing data and application logic for data processing;
  - **V**iew: Display the results from model;
  - **C**ontroller: It is between the Model and View. It takes input from the user, provides handlers of the inputs (simple processing), and let the model to process more complex functions.

J2EE's MVC Architecture



Model, View, and Controller in ASP .Net

- **View**: A directory contains the components that will generate the application's display (GUI) or the presentation layer of the application (output part).
- **Controller**: A directory contains the components that handle user inputs, work with the model, and ultimately select a view to render the displays in GUI.
- **Model**: A directory contains objects that that implement the part of the application logic related to the application's data domain. Often, model objects retrieve and store model state in a date sets, files, and database.

Automatically Generate Unit Test

http://msdn.microsoft.com/en-us/library/dd410597%28v=vs.100%29.aspx

- In the project, right-click the **Controllers** folder, click **Add**, and then click **Class**.
- In the **Name** text box, type **MapsControllerTest**.
- Click **Add**.

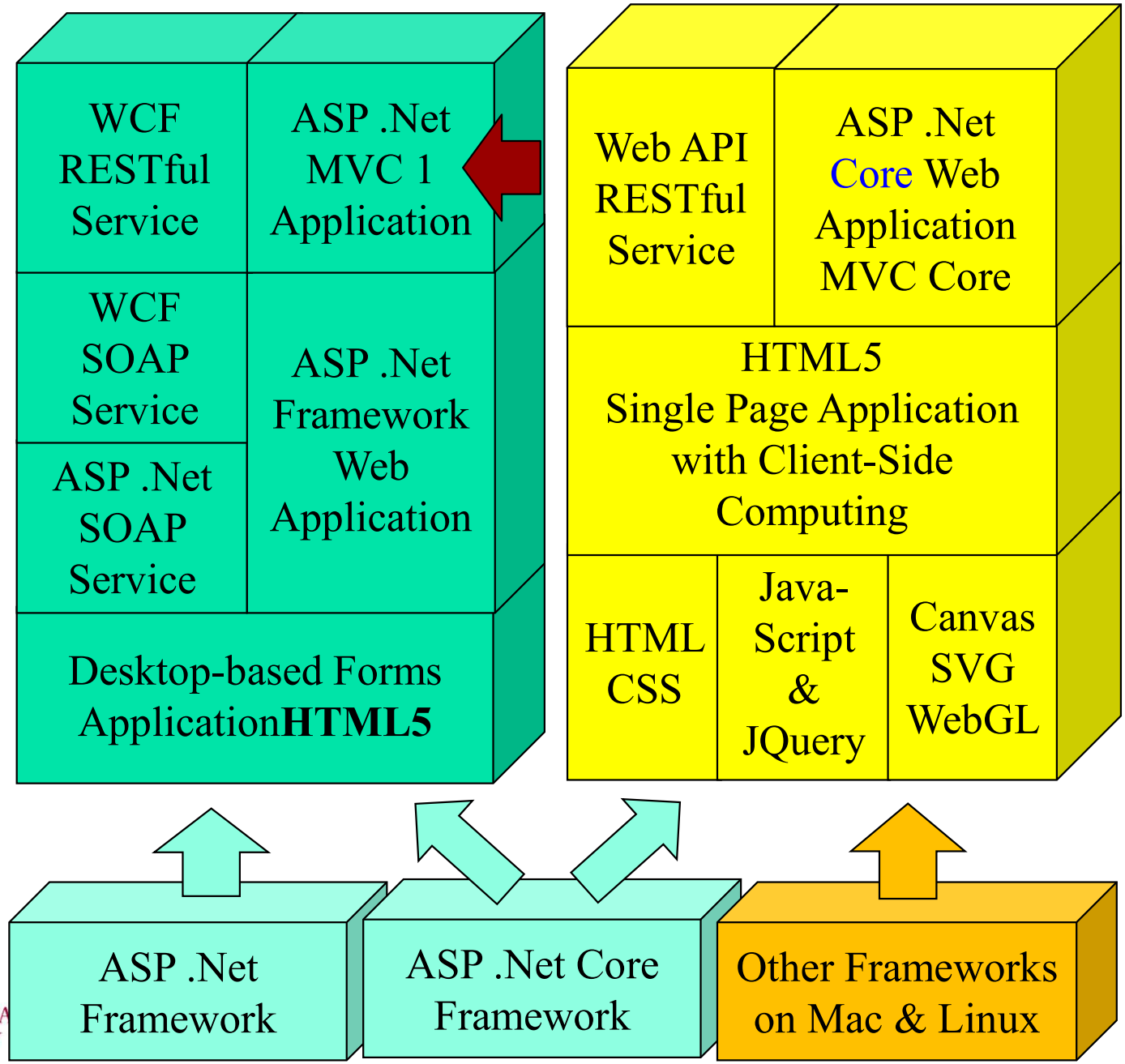
```
namespace MvcBasicWalkthrough.Tests.Controllers {
    [TestClass]
    public class MapsControllerTest {
        [TestMethod] public void ViewMaps() {
            // Arrange
            MapsController controller = new MapsController();
            // Act
            ViewResult result = controller.ViewMaps() as ViewResult;
            // Assert
            Assert.IsNotNull(result);
} }  }
```

Benefits of MVC Architecture

The loosely coupled feature (compared with ASP .Net architecture) among the tiers in Views, Models, and Controllers:

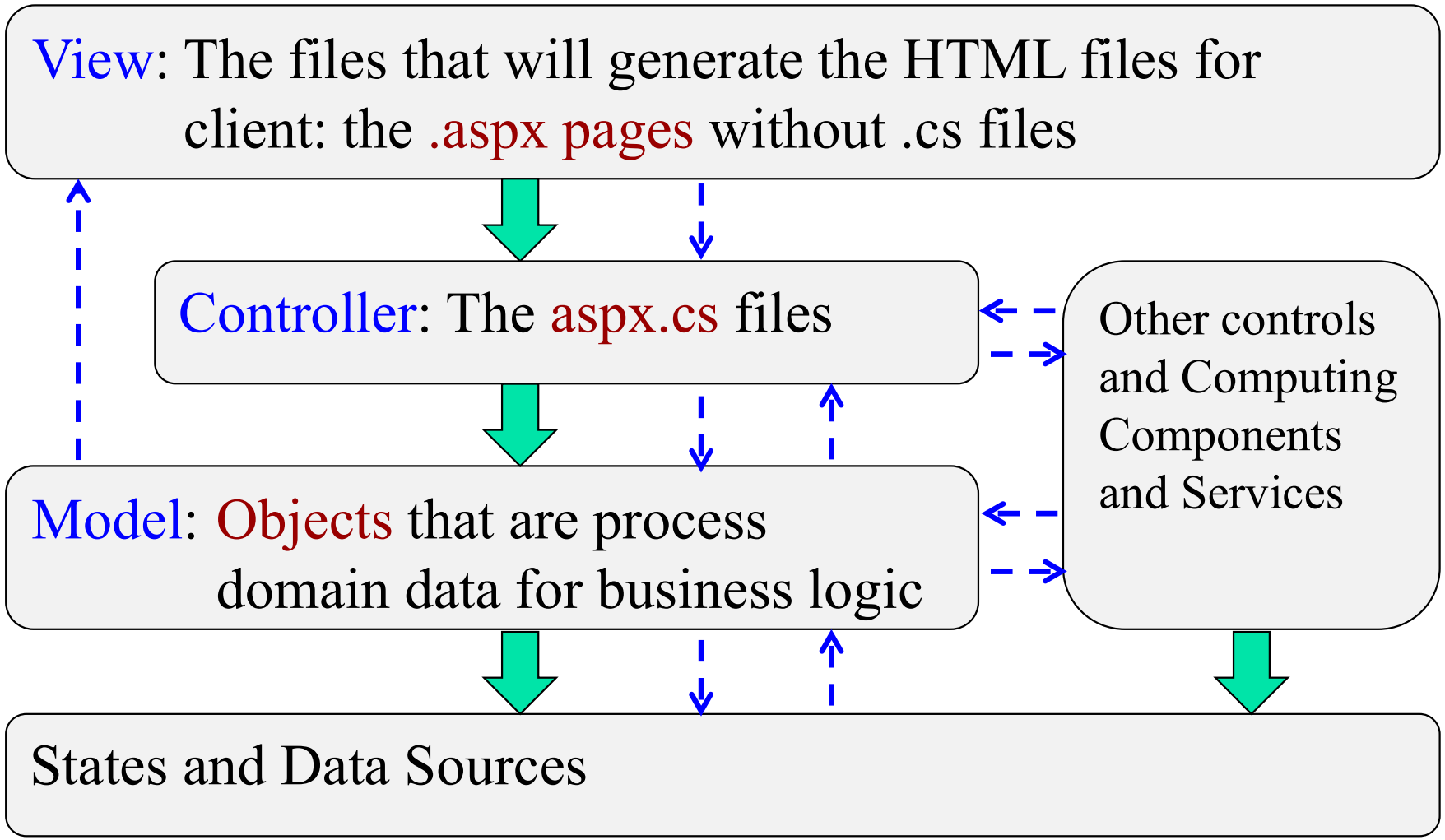
- Allow team members to develop different parts of the application with reduced interactions: GUI and code are separated;
- The template can generate unit tests by automatically adding mock objects (simulated objects) that mimic the behavior of other objects in controlled ways in the application;
- The restricted use of the view state, page postback, and related server controls, such as Gridview, allows the framework to render the XHTML page less dependent on the server features;
- The MVC applications are more friendly with URI-based access used for RESTful services, the feed data formats, and cloud computing environment (Unit 5).

The Evaluation Web Technologies



MVC in ASP .Net: A Four Tier Architecture

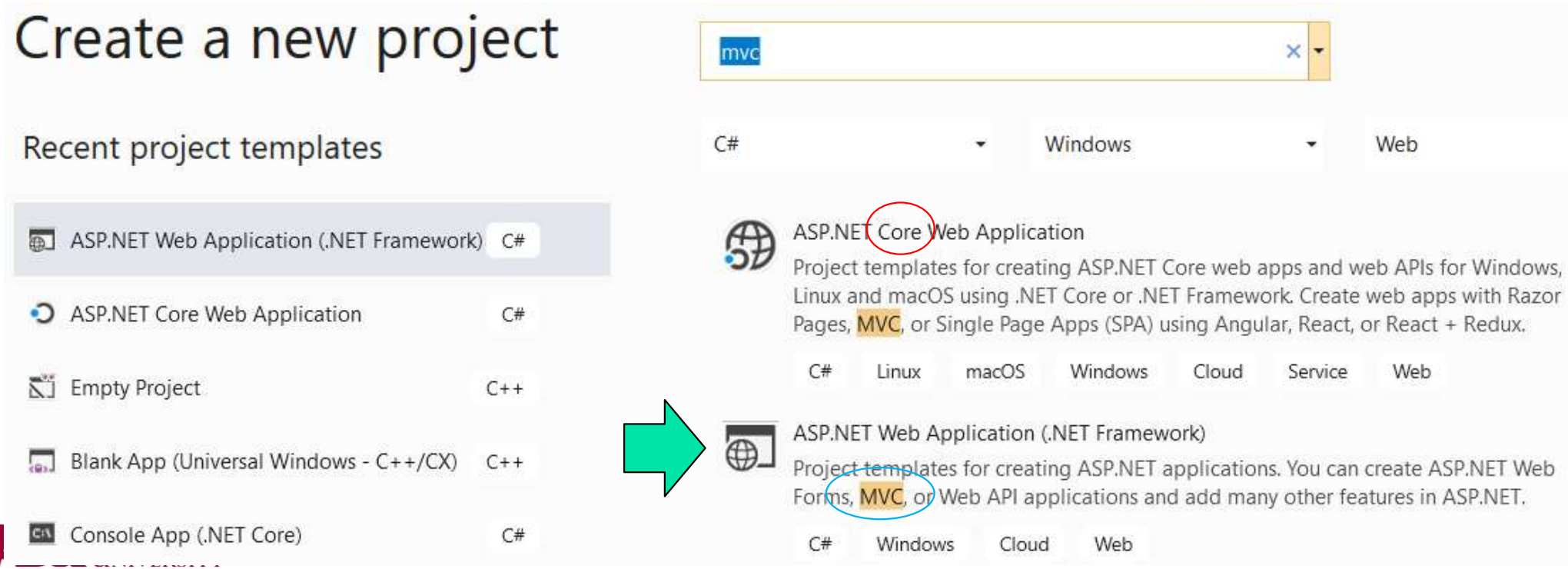
with loosely coupling among the tiers



ASP .Net MVC Architecture (MVC 1)

Reading: Text Chapter 5, Section 5.7

- ASP .Net offers MVC as an alternative project template;
- It is available for Web Apps, Mobile Apps, and Cloud Apps, without using any View State!
- Once selected the MVC template, the main components of the application will be organized in three directories: Models, Views, and Controllers.



MVC can be applied for developing different apps

Configure your new project

ASP.NET Web Application (.NET Framework) C# Windows Cloud Web

Project name

MvcApplication1

Location

C:\Users\ychen10\Dropbox (ASU)\Current ir

Create a new ASP.NET Web Application

Empty

An empty project template for creating ASP.NET applications. This template does not have any content in it.

Web Forms

A project template for creating ASP.NET Web Forms applications. ASP.NET Web Forms lets you build dynamic websites using a familiar drag-and-drop, event-driven model. A design surface and hundreds of controls and components let you rapidly build sophisticated, powerful UI-driven sites with data access.

MVC

A project template for creating ASP.NET MVC applications. ASP.NET MVC allows you to build applications using the Model-View-Controller architecture. ASP.NET MVC includes many features that enable fast, test-driven development for creating applications that use the latest standards.

Web API

A project template for creating RESTful HTTP services that can reach a broad range of clients including browsers and mobile devices.

Single Page Application

A project template for creating rich client side JavaScript driven HTML5 applications using ASP.NET Web API. Single Page Applications provide a rich user experience which includes client-side interactions using HTML5, CSS3, and JavaScript.

An MVC 1 Project: Organized Differently

Index.aspx | HomeController.cs

MvcApplication1 | MvcApplication1.Controller: | Contact()

```
using System.Web;
using System.Web.Mvc;

namespace MvcApplication1.Controllers
{
    0 references
    public class HomeController : Controller
    {
        0 references
        public ActionResult Index()
        {
            ViewBag.Message = "Modify this template to jump-start your

            return View();
        }

        0 references
        public ActionResult About()
        {
            ViewBag.Message = "Your app descripti

            return View();
        }

        0 references
```

100 %

Output

Solution Explorer

Search Solution Explorer (Ctrl+;)

- MvcApplication1
  - Properties
  - References
  - App_Data
  - App_Start
  - Content
  - Controllers
    - AccountController.cs
    - HomeController.cs
  - Filters
  - Images
  - Models
    - AccountModels.cs
  - Scripts
  - Views
    - Account
    - Home
      - About.aspx
      - Contact.aspx
      - Index.aspx
    - Shared
  - Web.config

.aspx.cs files, are moved to here

Put any business logic not related I/O are here

.aspx files, without .aspx.cs files

Add my Controller

Right click

Content
Controllers
AccountController.cs
HomeController.cs

View in Browser (Google Chrome) Ctrl+Shift+W
Browse With...
Add
Scope to This
New Solution Explorer View
Show on Code Map
Exclude From Project
Cut Ctrl+X
Copy Ctrl+C
Paste Ctrl+V
Delete Del
Rename
Open Folder in File Explorer
Properties Alt+Enter

Controller...
New Item... Ctrl+Shift+A
Existing Item... Shift+Alt+A
New Folder
Add ASP.NET Folder
Class...

Add Controller

Controller name:
MyUserControl1

Scaffolding options
Template:
Empty MVC controller

Empty MVC controller
MVC controller with read/write actions and views, using Entity Framework
MVC controller with empty read/write actions
Empty API controller
API controller with read/write actions, using Entity Framework
API controller with empty read/write actions

Views:
None
Advanced Options...

Add
Cancel

New Items are Added to the MVC 1 Project

ViewPageStaff.aspx | MyUserControl1.cs

Server Objects & Events | (No Events)

```
<%@ Page Language="C#" Inherits="System.Web.Mvc.Vie

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transi

<html xmlns="http://www.w3.org/1999/xhtml" >
<head runat="server">
    <title></title>
</head>
<body>
    <div>

    </div>
</body>
</html>
```

Solution Explorer

- Solution 'ASPNetMcvWebApp1' (2 projects)
  - ASPNetMcvWebApp1
    - Properties
    - References
    - App_Data
    - Content
    - Controllers
      - AccountController.cs
      - HomeController.cs
      - MyUserControl1.cs
    - Models
      - AccountModels.cs
    - Scripts
    - Views
      - Account
        - ChangePassword.aspx
        - ChangePasswordSuccess.aspx
        - LogOn.aspx
        - Register.aspx
      - Home
        - About.aspx
        - Index.aspx
        - ViewPageStaff.aspx
      - Shared
      - MyViewUserControl1.ascx
      - MyWebUserControl.ascx
        - MyWebUserControl.ascx.cs
        - MyWebUserControl.ascx.designer.cs
    - Web.config
    - Global.asax
    - Web.config

MVC user control's .cs file is added separately here.

A View Page is added here

A MVC user control. Only .ascx file is added

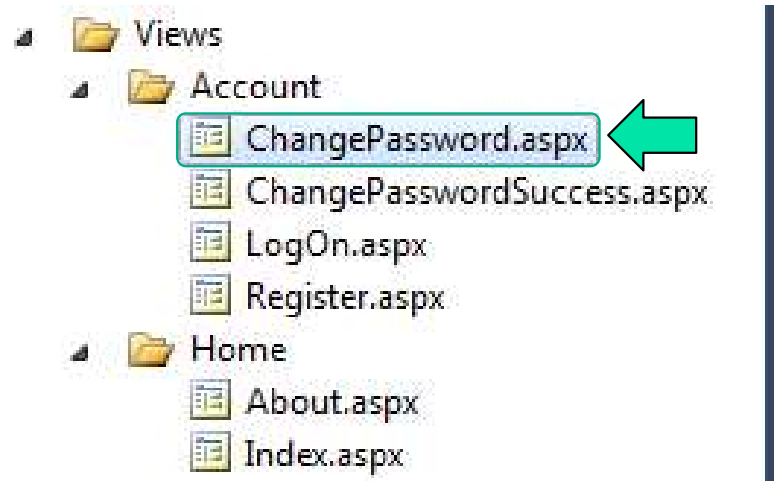
A traditional User control with both .ascx and .cs files

How are View and Controller Connected?

- In ASP .Net Framework, the .aspx files and .aspx.cs files are placed together in the same folder. The links are implicit. You must double-click the buttons to create the implicit links. It works because the two pages are tied together.
- **In MVC, the links** are explicit.
  - Each View component (.aspx file) in browser contains a list of JavaScript calls.
  - The Controller of the component contains the methods to be called. View page and .cs page can be completely separated.

ChangePassword in ChangePassword.aspx, making JS calls

```
<%@ Page Language="C#" MasterPageFile="~/Views/Shared/Site.Master" Inherits="System.Web.Mvc.ViewPage<MvcApp.Models.ChangePasswordModel>"

<asp:Content ID="changePasswordTitle" ContentPlaceHolderID="TitleContent" runat="server">
    Change Password
</asp:Content>
<asp:Content ID="changePasswordContent" ContentPlaceHolderID="MainContent" runat="server">
    <h2>Change Password</h2>
    <p> Use the form below to change your password. </p>
    <p> New passwords are required to be a minimum of <%: Membership.MinRequiredPasswordLength %> characters in length. </p>
    <script src="<%: Url.Content("~/Scripts/jquery.validate.min.js") %>" type="text/javascript"></script>
    <script src="<%: Url.Content("~/Scripts/jquery.validate.unobtrusive.min.js") %>" type="text/javascript"></script>
    <% using (Html.BeginForm()) { %>
        <%: Html.ValidationSummary(true, "Password change was unsuccessful. Please correct the errors and try again.") %>
        <div>
            <fieldset>
                <legend>Account Information</legend>
                <div class="editor-label">
                    <%: Html.LabelFor(m => m.OldPassword) %>
                </div>
                <div class="editor-field">
                    <%: Html.PasswordFor(m => m.OldPassword) %>
                    <%: Html.ValidationMessageFor(m => m.OldPassword) %>
                </div>
                <div class="editor-label">
                    <%: Html.LabelFor(m => m.NewPassword) %>
                </div>
                <div class="editor-field">
                    <%: Html.PasswordFor(m => m.NewPassword) %>
                    <%: Html.ValidationMessageFor(m => m.NewPassword) %>
                </div>
                <div class="editor-label">
                    <%: Html.LabelFor(m => m.ConfirmPassword) %>
                </div>
                <div class="editor-field">
                    <%: Html.PasswordFor(m => m.ConfirmPassword) %>
                    <%: Html.ValidationMessageFor(m => m.ConfirmPassword) %>
```



ChangePassword method in AccountController.cs

To be called from JavaScript

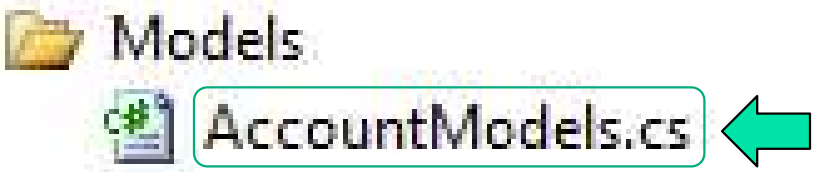
```
// POST: /Account/ChangePassword
    [Authorize]
    [HttpPost]
    public ActionResult ChangePassword(ChangePasswordModel model)  {
        if (ModelState.IsValid)  {
            // ChangePassword will throw an exception rather than return false in certain failure scenarios.
            bool changePasswordSucceeded;
            try  {
                MembershipUser currentUser = Membership.GetUser(User.Identity.Name, true /* userIsOnline */);
                changePasswordSucceeded = currentUser.ChangePassword(model.OldPassword, model.NewPassword);
            }
            catch (Exception){
                changePasswordSucceeded = false;
            }
            if (changePasswordSucceeded)  {
                return RedirectToAction("ChangePasswordSuccess");
            }
            else  {
                ModelState.AddModelError("", "The current password is incorrect or the new password is invalid.");
            }
        }
        // If we got this far, something failed, redisplay form
        return View(model);
    }
```

To call function in Model

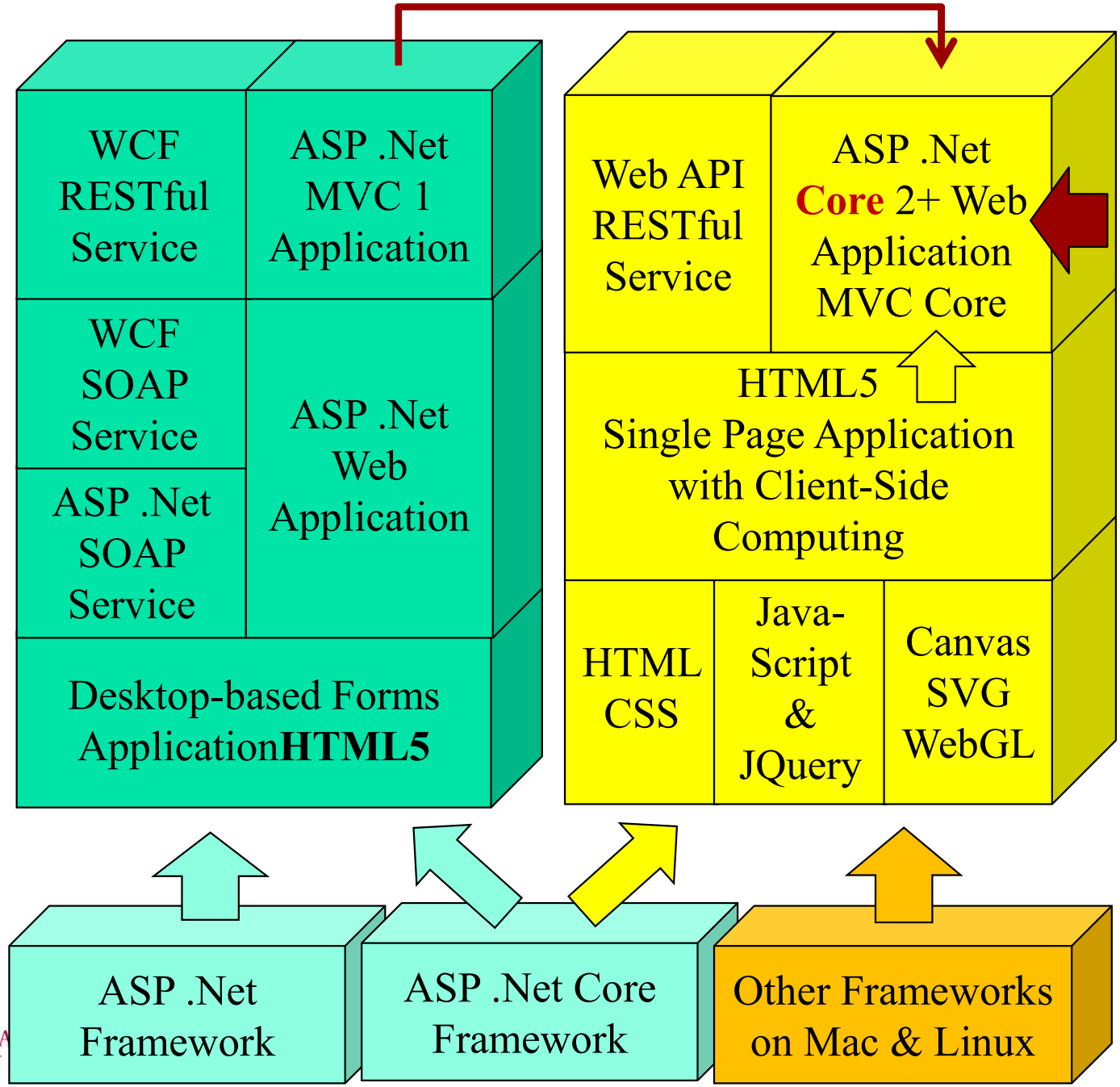
ChangePassword method in AccountModels.cs

```
public bool ChangePassword(string userName, string oldPassword, string newPassword)   {
        if (String.IsNullOrEmpty(userName)) throw new ArgumentException("Value cannot be
null or empty.", "userName");
        if (String.IsNullOrEmpty(oldPassword)) throw new ArgumentException("Value cannot be
null or empty.", "oldPassword");
        if (String.IsNullOrEmpty(newPassword)) throw new ArgumentException("Value cannot be
null or empty.", "newPassword");
        // The underlying ChangePassword() will throw an exception rather
        // than return false in certain failure scenarios.
        try  {
            MembershipUser currentUser = _provider.GetUser(userName, true /* userIsOnline */);
            return currentUser.ChangePassword(oldPassword, newPassword);
        }
        catch (ArgumentException) {
            return false;
        }
        catch (MembershipPasswordException)  {
            return false;
        }
}
```

Access database



From MVC 1 to MVC 2 (or later version)

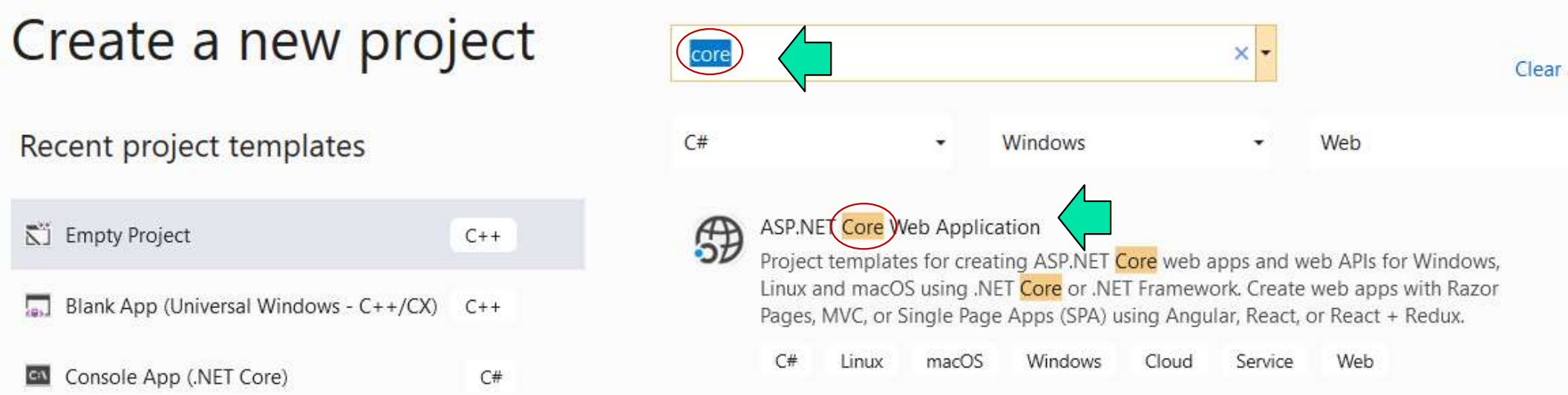


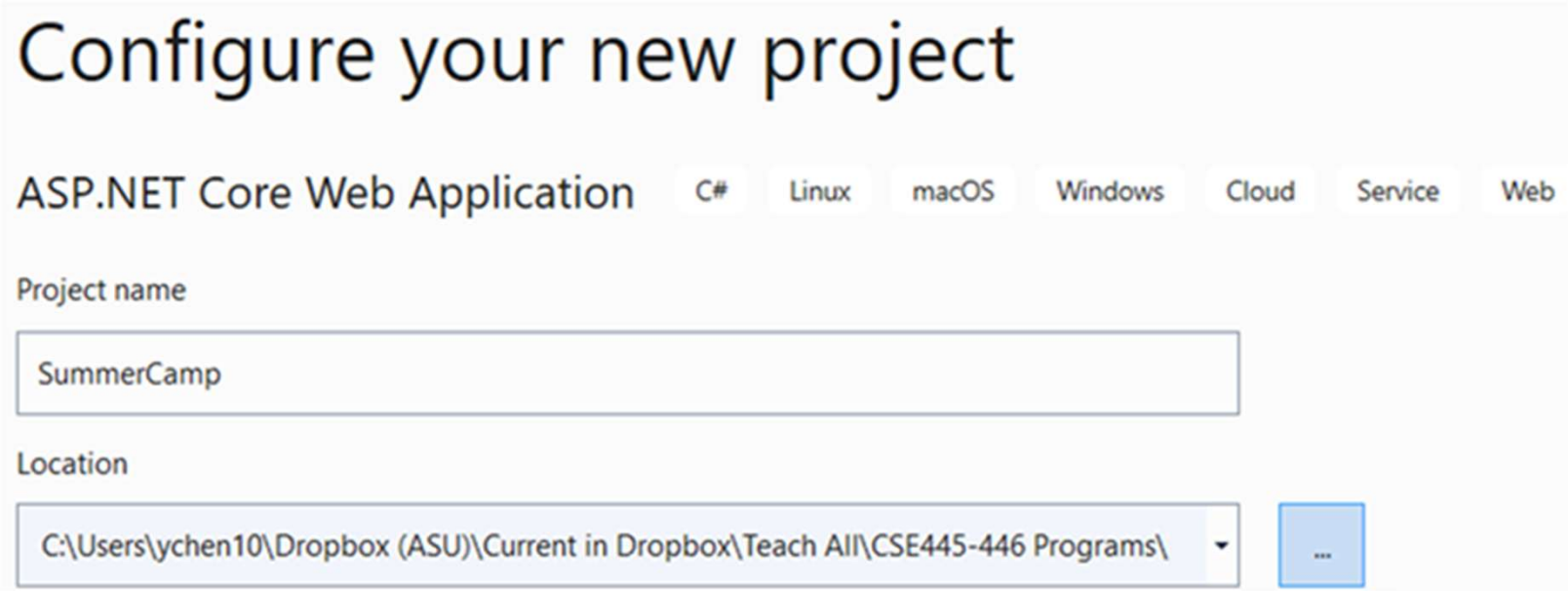
Case Study: Summer Camp Registration

https://venus.sod.asu.edu/WSRepository/MVC/SummerCampMVC/Home/Index

- Create an ASP .Net **Core** Web Application
- Create the main page for information
- Inviting for enrollment
- Signup form
- Store student list in a List structure
- Showing Controllers → View
- Showing Controllers → Model

Getting Started with MVC on ASP .Net Core





Creating a NewASP .Net Core Project

Create a new ASP.NET Core web application

.NET Core | ASP.NET Core 3.1

Empty

An empty project template for creating an ASP.NET Core application. This template does not have any

API

A project template for creating an ASP.NET Core application with an example Controller for a RESTful
This template can also be used for ASP.NET Core MVC Views and Controllers.

Web Application

A project template for creating an ASP.NET Core application with example ASP.NET Razor Pages conte

Web Application (Model-View-Controller)

A project template for creating an ASP.NET Core application with example ASP.NET Core MVC Views a
Controllers. This template can also be used for RESTful HTTP services.

Solution Explorer

Search Solution Explorer (Ctrl+;)

- SummerCamp
  - Connected Services
  - Dependencies
  - Properties
  - wwwroot
  - Controllers
    - HomeController.cs
  - Models
    - ErrorViewModel.cs
  - Views
    - Home
    - Shared
    - _ViewImports.cshtm
    - _ViewStart.cshtml
  - appsettings.json
  - Program.cs

MVC Project in ASP .Net Core

HomeController.cs | SummerCamp

SummerCamp | SummerCamp.Controllers.Hom | Index()

```
3   using System.Diagnostics;
4   using System.Linq;
5   using System.Threading.Tasks;
6   using Microsoft.AspNetCore.Mvc;
7   using SummerCamp.Models;
8
9   namespace SummerCamp.Controllers
10  {
        0 references
11      public class HomeController : Controller
12      {
            0 references
13          public IActionResult Index()
14          {
15              return View();
16          }
17
            0 references
18          public IActionResult About()
19          {
20              ViewData["Message"] = "Your application description page
21
22              return View();
23          }
24
            0 references
25          public IActionResult Contact()
26          {
27              ViewData["Message"] = "Your contact
```

MVC Controllers with .cs files

Models with .cs files

MVC Views with .cshtml files for html and binding with models:

```
@{ ViewData["Title"]
= "Home Page"; }
```

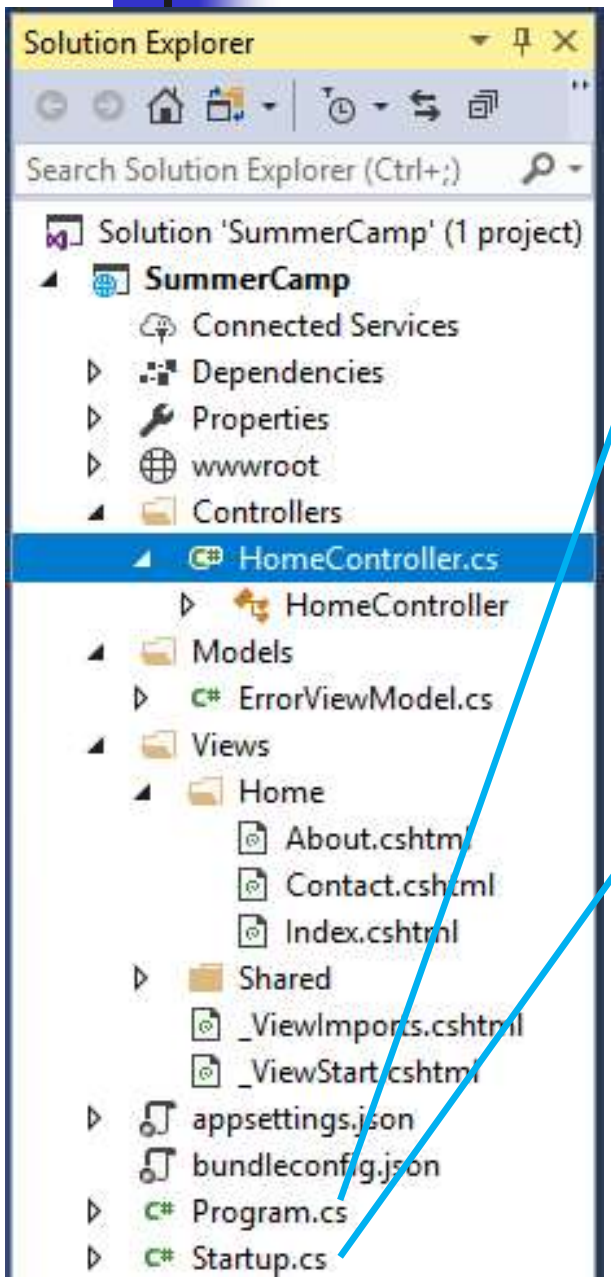
Solution Explorer

Search Solution Explorer (Ctrl+;)

- Solution 'SummerCamp' (1 project)
  - SummerCamp
    - Connected Services
    - Dependencies
    - Properties
    - wwwroot
    - Controllers
      - HomeController.cs
        - HomeController
    - Models
      - ErrorViewModel.cs
    - Views
      - Home
        - About.cshtml
        - Contact.cshtml
        - Index.cshtml
      - Shared
      - _ViewImports.cshtml
      - _ViewStart.cshtml
    - appsettings.json
    - bundleconfig.json
    - Program.cs
    - Startup.cs

Program Starting Point

System Generated



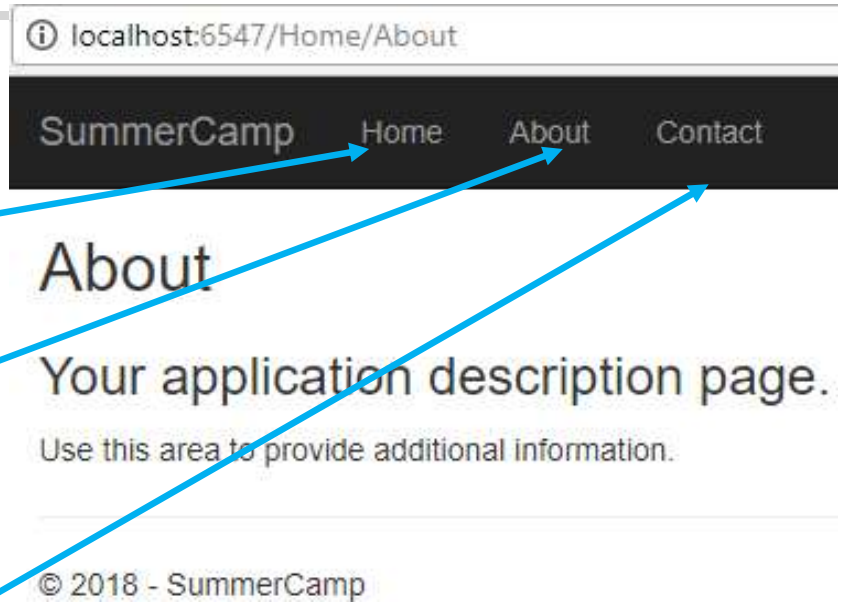

```
namespace SummerCamp
{
    0 references
    public class Program
    {
        0 references
        public static void Main(string[] args)
        {
            BuildWebHost(args).Run();
        }

        1 reference
        public static IWebHost BuildWebHost(string[] args) =>
            WebHost.CreateDefaultBuilder(args)
                .UseStartup<Startup>()
                .Build();
    }
}
```

Main() method is the program starting point

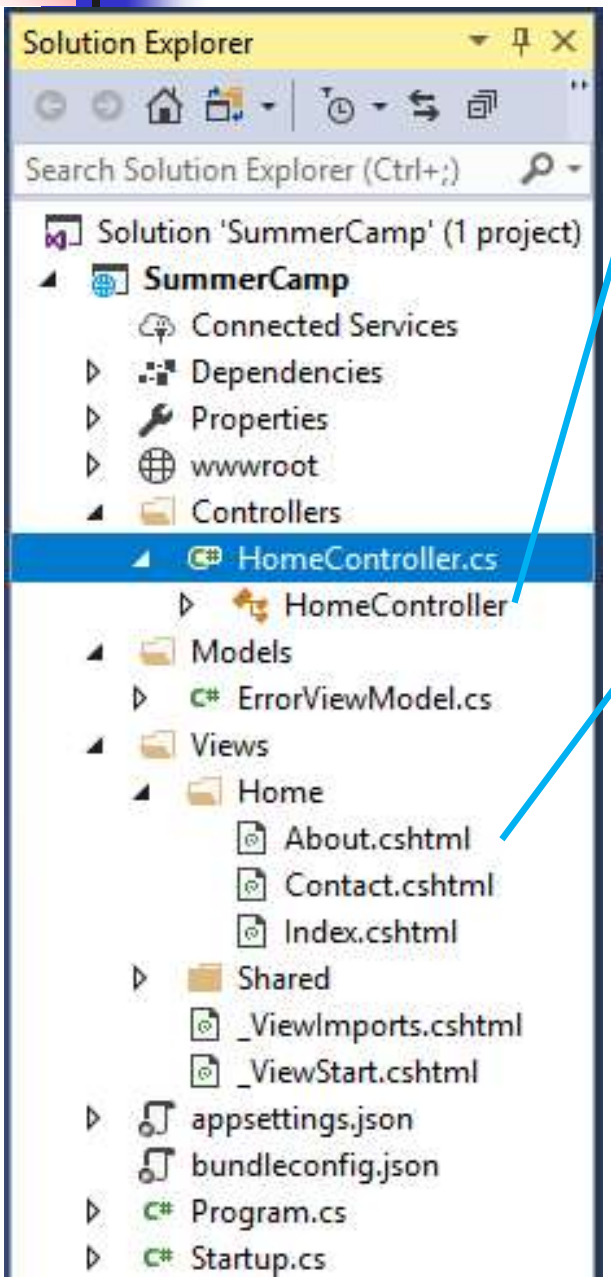
```
public Startup(IConfiguration configuration)...

1 reference
public IConfiguration Configuration { get; }

// This method gets called by the runtime. Use this method to add servic
0 references
public void ConfigureServices(IServiceCollection services)...

// This method gets called by the runtime. Use this method to configure
0 references
public void Configure(IApplicationBuilder app, IHostingEnvironment env)
{
    if (env.IsDevelopment())
    {
        app.UseBrowserLink();
        app.UseDeveloperExceptionPage();
    }
    else { app.UseExceptionHandler("/Home/Error"); }

    app.UseStaticFiles();

    app.UseMvc(routes =>
    {
        routes.MapRoute(
            name: "default",
            template: "{controller=Home}/{action=Index}/{id?}");
    });
```

HomeController: System Generated

```
namespace SummerCamp.Controllers
{
    0 references
    public class HomeController : Controller
    {
        0 references
        public IActionResult Index()
        {
            return View();
        }

        0 references
        public IActionResult About()
        {
            ViewData["Message"] = "Your application description page.";
            return View();
        }
        2
        0 references
        public IActionResult Contact()
        {
            ViewData["Message"] = "Your contact page.";
            return View();
        }

        0 references
        public IActionResult Error()
        {
            return View(new ErrorViewModel { RequestId = Activity.Current?.Id ?? HttpContext.Trace
        }
    }
}
```



Correlation Between Controller and View

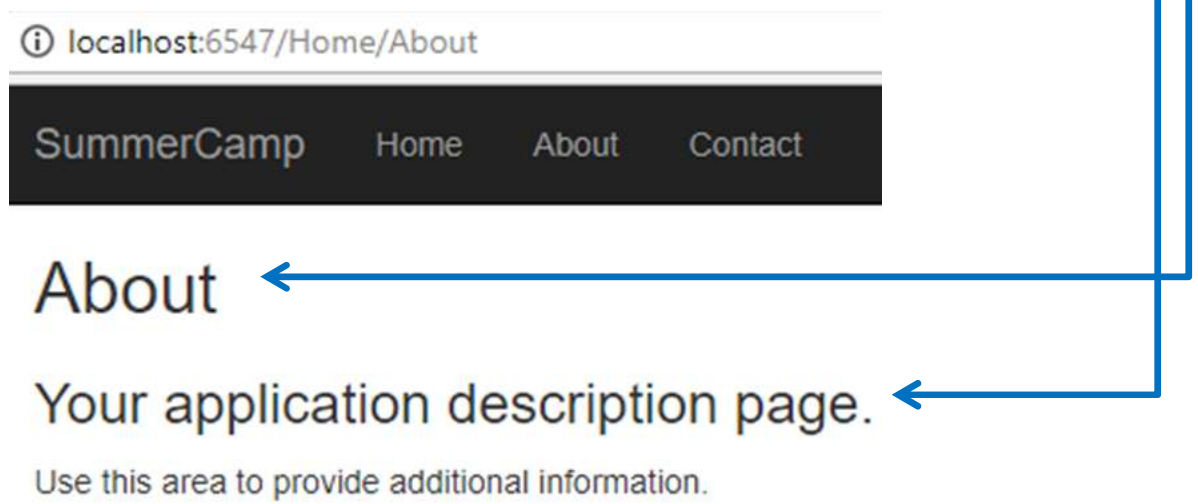



```
public IActionResult About()
{
    ViewData["Message"] = "Your application description page.";
    return View();
}
```

Define "Title"

About.cshtml* | Index.cshtml | Program.cs | Startup.cs | Hom

```
1  @{
2      ViewData["Title"] = "About";
3  }
4  <h2>@ViewData["Title"]</h2>
5  <h3>@ViewData["Message"]</h3>
6
7  <p>Use this area to provide additional information.</p>
```



Modify the Index.cshtml Page

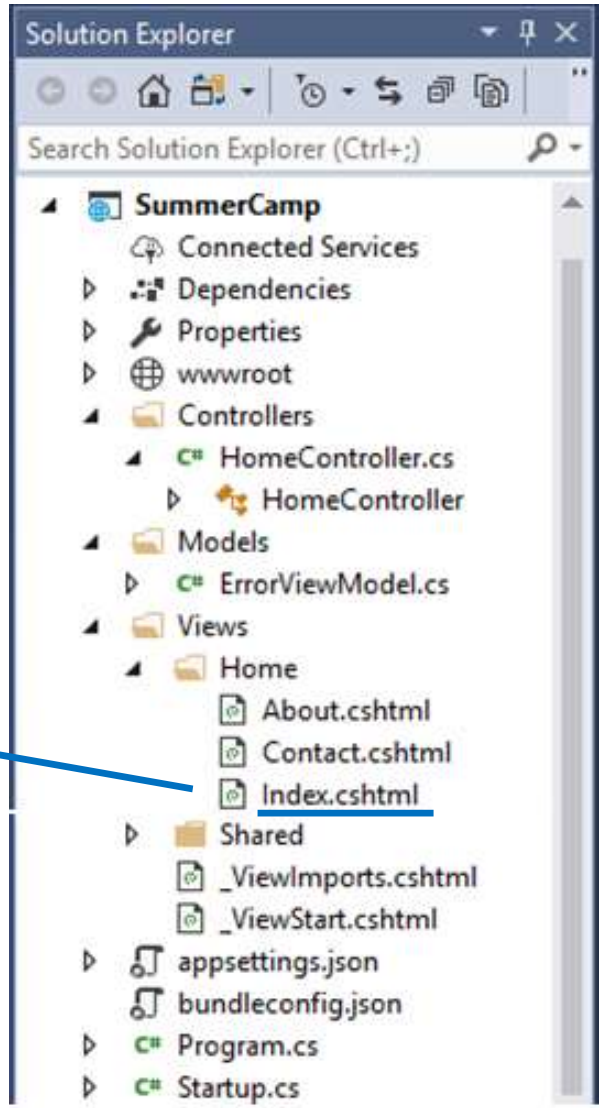
- Write your code in Index page:

Index.cshtml | HomeController.cs | StudentList.cs | Confirmation.cshtml

```
    Layout = null;
}
<!DOCTYPE html>
<html>
<head>
    <meta name="viewport" content="width=device-width" />
    <title>Semester</title>
    <link rel="stylesheet" href="/lib/bootstrap/dist/css/bootstrap.css" />
</head>
<body>
    <div class="text-center">
        <h3>Hello high school students, it is @ViewBag.Semester!</h3>
        <h3>We are going to offer a summer robotics camp</h3>
        <h4>You can sign up the camp now:</h4>
        <a class="btn btn-primary" asp-action="SignupForm">Sign Up Now</a>
    </div>
</body>
</html>
```

Define a variable

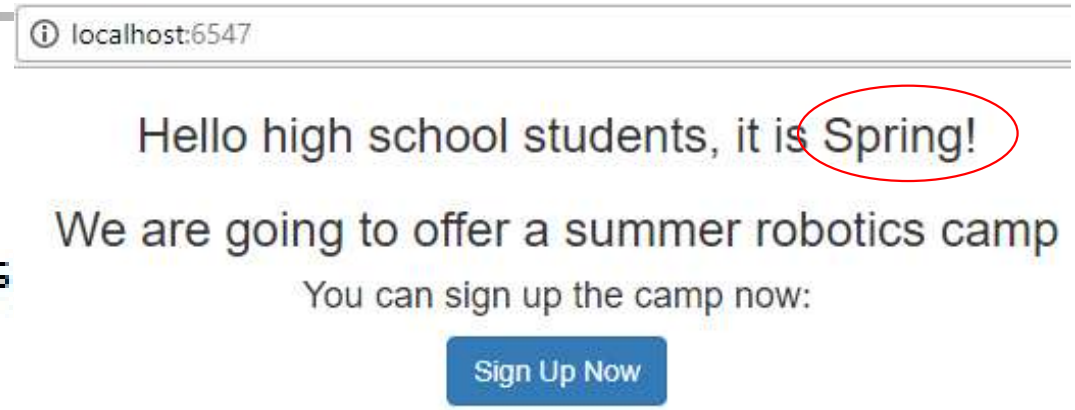
A page to be added

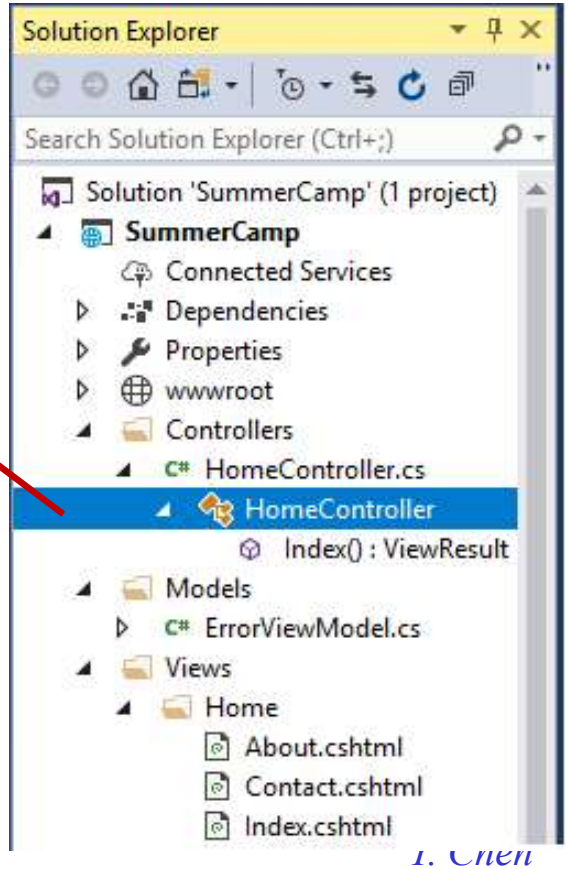


Modify HomeController.cs

- Link the code to the SemesterView page

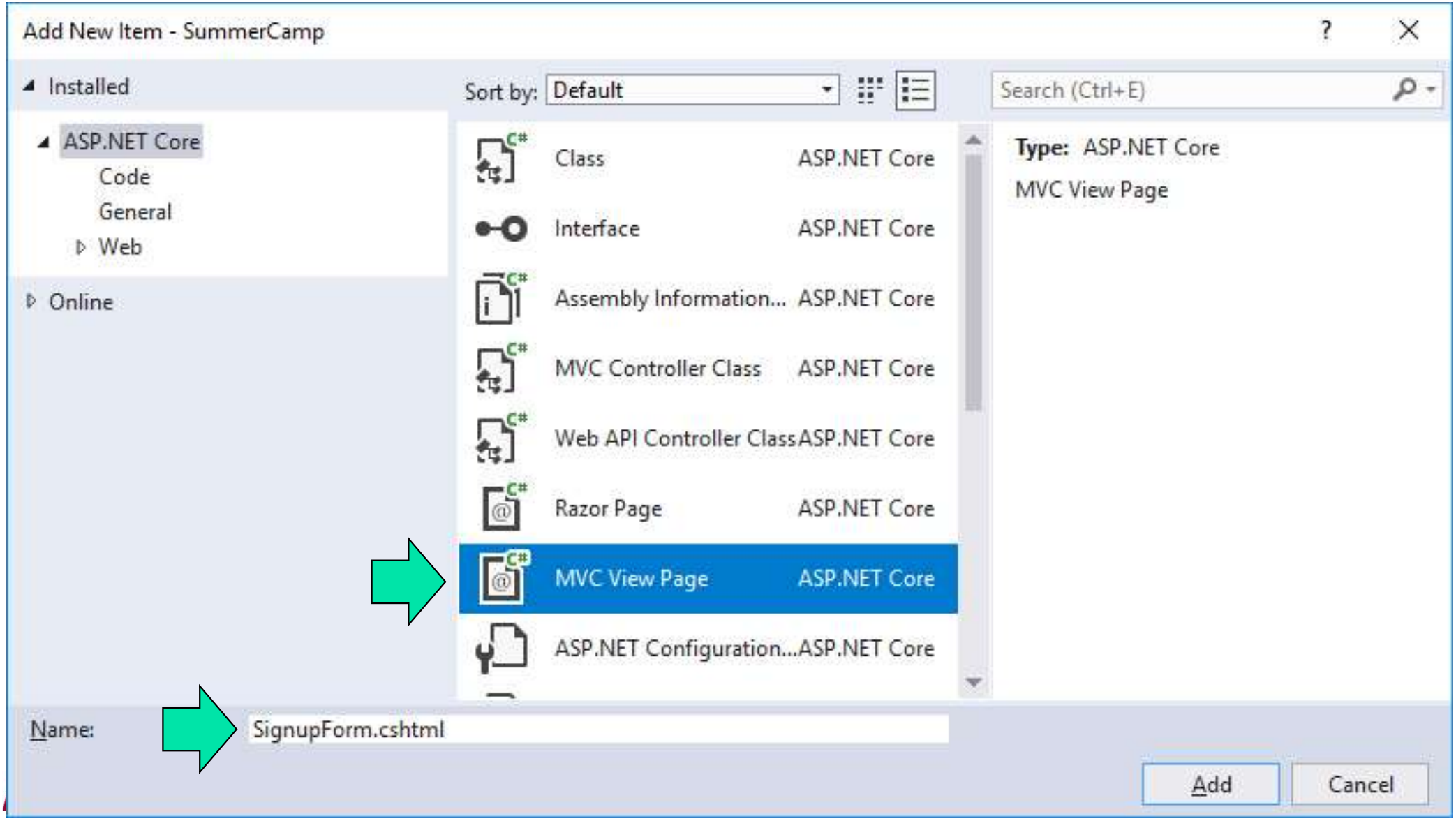
```
namespace SummerCamp.Controllers
{
    0 references
    public class HomeController : Controller
    {
        0 references
        public ViewResult Index()
        {
            int m = DateTime.Now.Month;
            if (m < 6)
                ViewBag.Semester = "Spring";
            else if (m < 8)
                ViewBag.Semester = "Summer";
            else if (m >= 8)
                ViewBag.Semester = "Fall";
            return View("Index");
        }
```





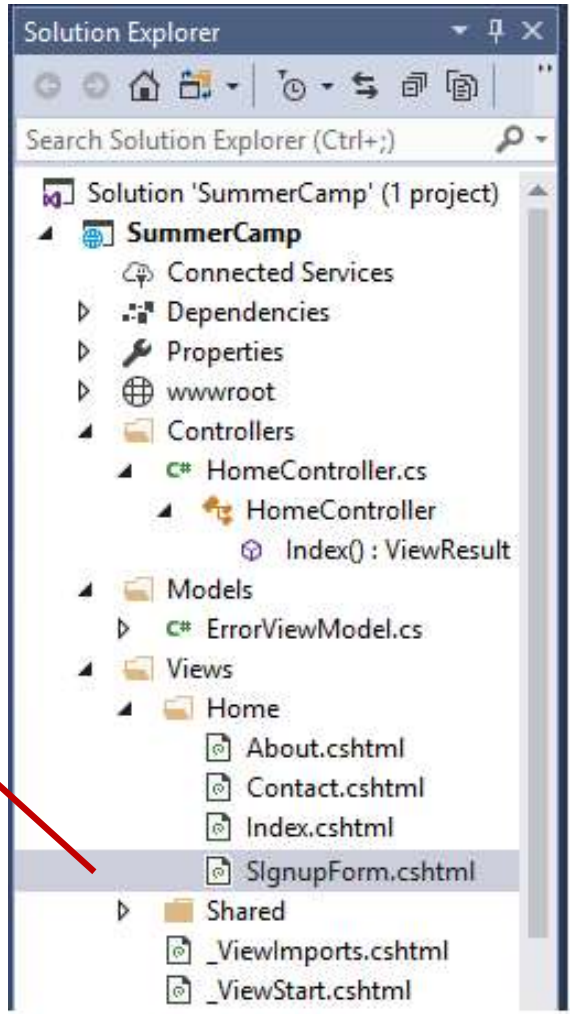
Add a New MVC View Page

- Right Views-Home and Choose Add → New Item:



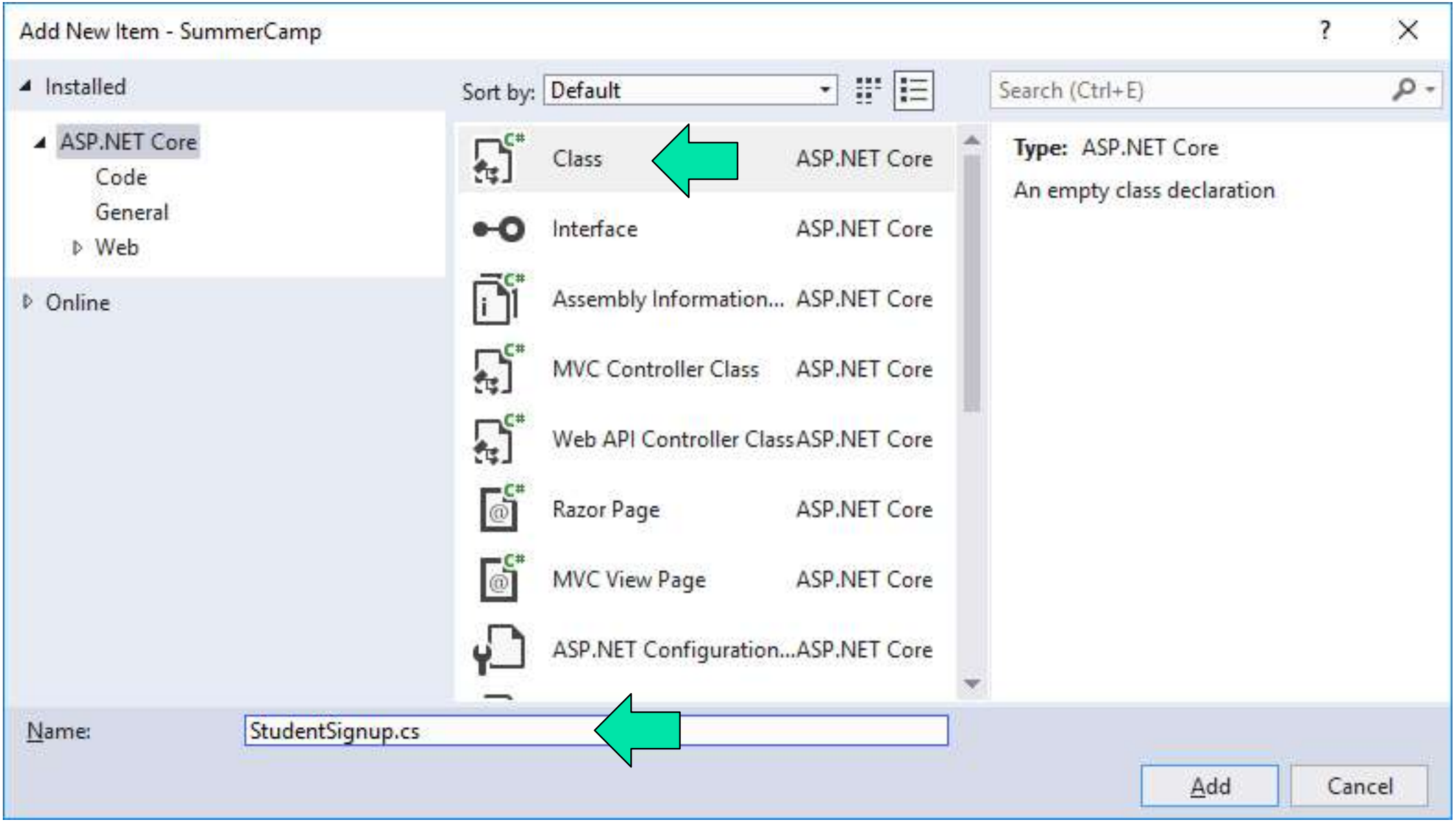
MVC View Page SignupForm Added

```
<body>
    <div class="panel panel-success">
        <div class="panel-heading text-center"><h4>RSVP</h4></div>
        <div class="panel-body">
            <form class="p-a-1" asp-action="SignupForm" method="post">
                <div asp-validation-summary="All"></div>
                <div class="form-group">
                    <label asp-for="Name">Your name:</label>
                    <input class="form-control" asp-for="Name" />
                </div>
                <div class="form-group">
                    <label asp-for="Email">Your email:</label>
                    <input class="form-control" asp-for="Email" />
                </div>
                <div class="form-group">
                    <label>Are you a high school student currently?</label>
                    <select class="form-control" asp-for="Signup">
                        <option value="">Option</option>
                        <option value="true">Yes, I am a high school student</option>
                        <option value="false">No, I am not a high school student</option>
                    </select>
                </div>
                <div class="text-center">
                    <button class="btn btn-primary" type="submit">
                        Submit the Form Now
                    </button>
                </div>
            </form>
        </div>
    </div>
</body>
```



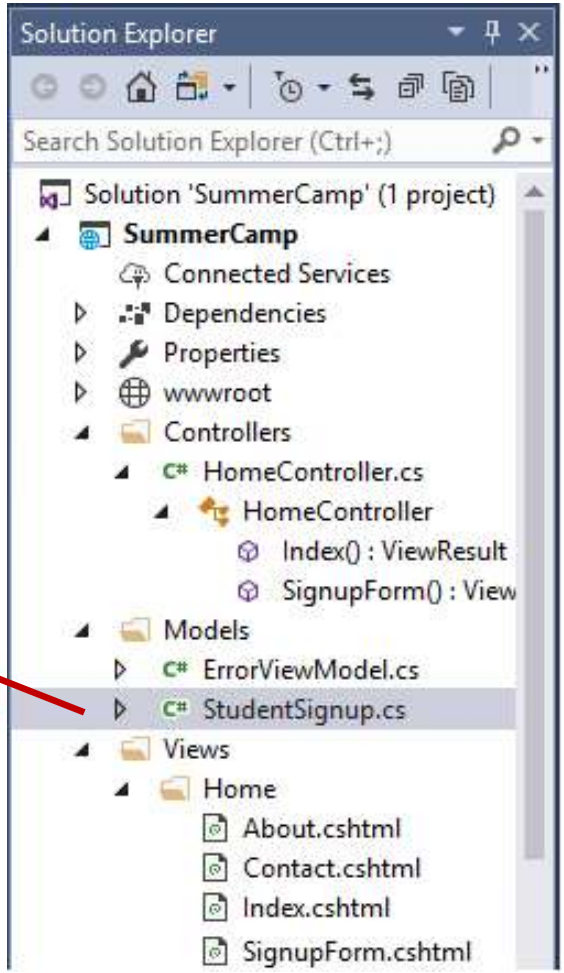
Add a Model to deal with Student Signup

- Right-click Project → Add → Class

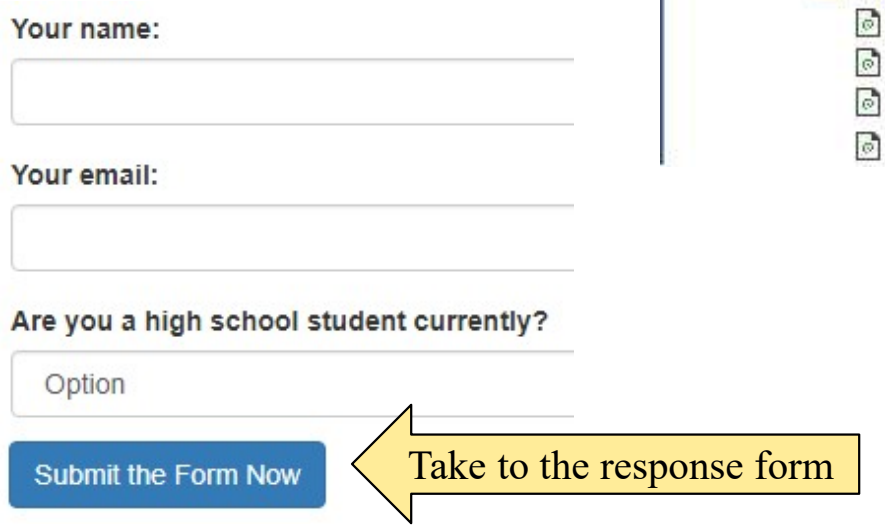


Define the Signup Information in Model Page

```
using System.ComponentModel.DataAnnotations;

namespace SummerCamp.Models
{
    4 references
    public class StudentSignup
    {
        [Required(ErrorMessage = "Please enter your name")]
        2 references
        public string Name { get; set; }

        [Required(ErrorMessage = "Please enter your email address")]
        [RegularExpression(".+\\@.+\\..+",
            ErrorMessage = "Please enter a valid email address")]
        2 references
        public string Email { get; set; }

        [Required(ErrorMessage = "Please let us know if you are a high school
        1 reference
        public bool? Signup { get; set; }
    }
}
```

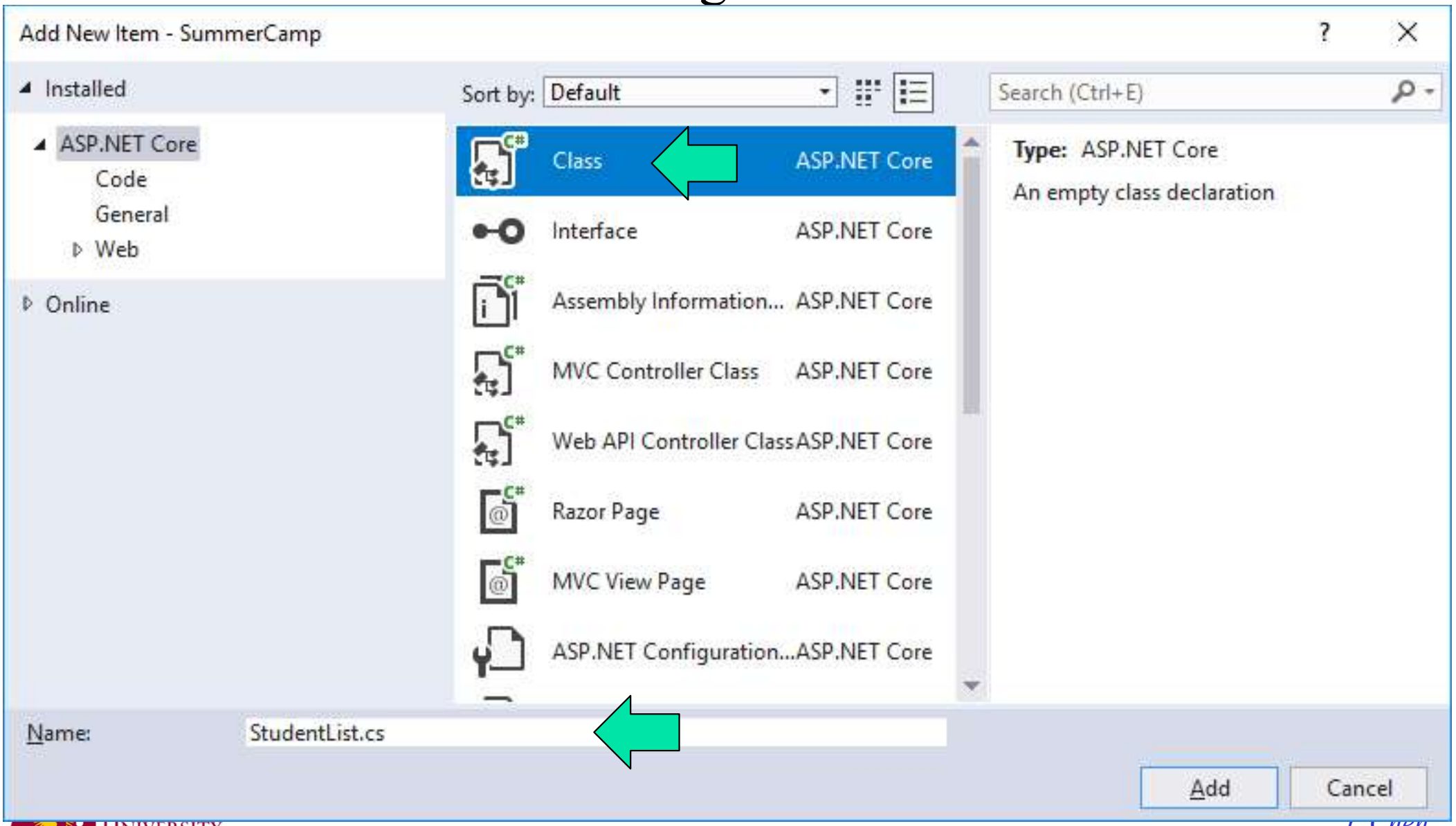


Start the Program and click Signup Now:



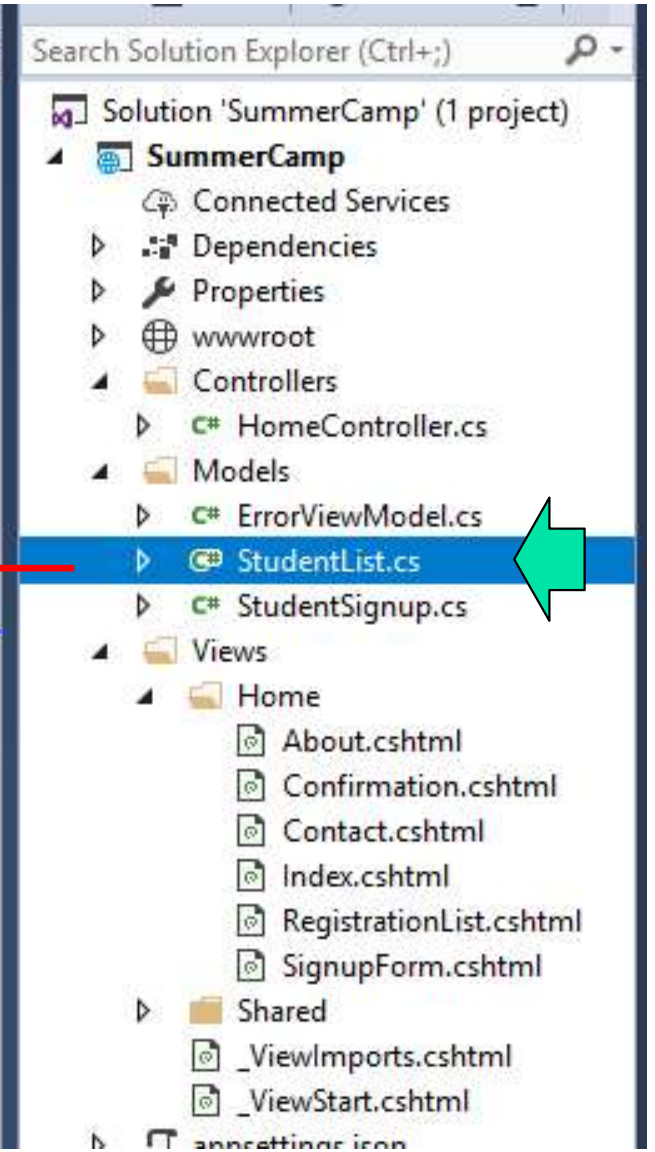
Receive & Store Signed up Student Information

- Add a Model for storing student list:



Use a List Type to Hold the Student List

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;

namespace SummerCamp.Models
{
    2 references
    public class StudentList
    {
        private static List<StudentSignup> mylist = new List<StudentSignup>();

        1 reference
        public static IEnumerable<StudentSignup> Responses
        {
            get
            {
                return mylist;
            }
        }

        1 reference
        public static void AddResponse(StudentSignup student)
        {
            mylist.Add(student);
        }
    }

}
```



Demo:

https://venus.sod.asu.edu/WSRepository/MVC/SummerCampMVC/Home/Index

Hello high school students, it is Spring!

We are going to offer a summer robotics camp

You can sign up the camp now:

Sign Up Now

Your name:

Mary Smith

Your email:

mary@asu.edu

Are you a high school student currently?

Yes, I am a high school student

Submit the Form Now

Thank you, Mary Smith!

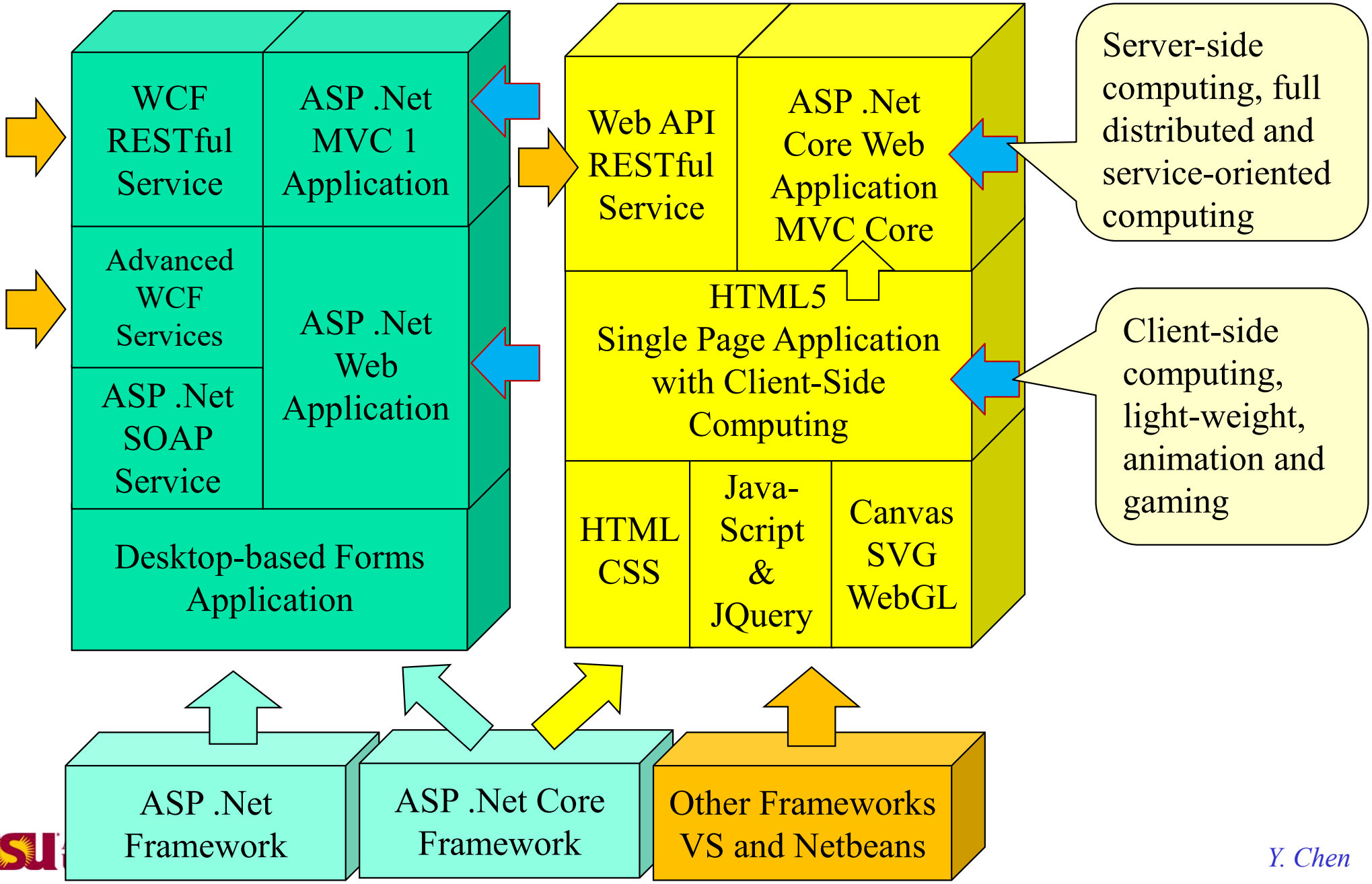
Thank you for signing up to the summer robotics camp

Click here to see who have signed up.

Here is the list of students who have signed up the summer robotics camp

Name	Email
Jone Doe	jonh@asu.edu
Mary Smith	mary@asu.edu

Summary


WCF RESTful Service
ASP .Net MVC 1 Application
Advanced WCF Services
ASP .Net SOAP Service
ASP .Net Web Application
Desktop-based Forms Application
Web API RESTful Service
ASP .Net Core Web Application MVC Core
HTML5 Single Page Application with Client-Side Computing
HTML CSS
Java-Script & JQuery
Canvas SVG WebGL
Server-side computing, full distributed and service-oriented computing
Client-side computing, light-weight, animation and gaming
ASP .Net Framework
ASP .Net Core Framework
Other Frameworks VS and Netbeans